

Homework 1

Thuong Hoang

Part 1

Problem 1

1. Since $p_i(x) = \mathcal{N}(\mu_i, \sigma_i^2)$ for $i = 1, 2$ with $\sigma_i > 0$ then

$p(x) = \frac{p_1(x)p_2(x)}{Z}$, $Z = \int_{-\infty}^{\infty} p_1(x)p_2(x)dx$ is non-negative because Gaussian is non-negative. We must now determine whether the total probability is equal to 1.

$$\int_{-\infty}^{\infty} p(x)dx = \int_{-\infty}^{\infty} \frac{p_1(x)p_2(x)}{Z}dx = \frac{1}{Z} \int_{-\infty}^{\infty} p_1(x)p_2(x)dx = \frac{1}{Z}Z = 1$$

With this, we know that $p(x)$ is non-negative and the total probability equates to 1.

To determine the parameters of $p(x)$, we must expand it.

$$\begin{aligned} p(x) &= p_1(x)p_2(x) \\ &= \frac{1}{2\pi\sigma_1} \frac{1}{2\pi\sigma_2} \exp\left(-\frac{(x-\mu_1)^2}{2\sigma_1^2} - \frac{(x-\mu_2)^2}{2\sigma_2^2}\right) \\ &= \frac{1}{2\pi\sigma_1\sigma_2} \exp\left(-\frac{1}{2}\left[\frac{(x-\mu_1)^2}{\sigma_1^2} + \frac{(x-\mu_2)^2}{\sigma_2^2}\right]\right) \end{aligned}$$

We know that $\frac{1}{2\pi\sigma_1\sigma_2}$ is a constant; hence, we can ignore it. Here we have

$(x-\mu)^2$. Doing the multiplication will get us $x^2 - 2x\mu + \mu^2$. Now, we substitute it back into the original equation to get this

$$\exp\left(-\frac{1}{2}\left[\frac{x^2 - 2x\mu + \mu^2}{\sigma_1^2} + \frac{x^2 - 2x\mu + \mu^2}{\sigma_2^2}\right]\right) = \exp\left(-\frac{1}{2}\left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right)x^2 - 2\left(\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2}\right)x + \frac{\mu_1^2}{\sigma_1^2} + \frac{\mu_2^2}{\sigma_2^2}\right)$$

Recall completion of squares, $ax^2 + bx + c$. The above result follows a variation of this format. It follows $\exp\left(\frac{1}{2}(ax^2 + bx + c)\right)$. From a Gaussian equation, we can do the following

$$\begin{aligned}
\frac{(x - \mu_1)^2}{2\sigma^2} &= \frac{x^2 - 2x\mu + \mu^2}{2\sigma^2} \\
&= \frac{x^2}{2\sigma^2} + \frac{\mu}{\sigma^2}x + \frac{\mu^2}{2\sigma^2} \\
&= \frac{1}{2} \left(\frac{x^2}{\sigma^2} - \frac{2\mu}{\sigma^2}x + \frac{\mu^2}{\sigma^2} \right)
\end{aligned}$$

Then we can identify $a = \frac{1}{\mu^2}$ and $b = -2\frac{\mu}{\sigma^2}$. Remember the original formula simplified to this:

$$\exp\left(-\frac{1}{2}\left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right)x^2 - 2\left(\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2}\right)x + C\right)$$

So, I can use completion of squares to do the following

$$\begin{aligned}
\frac{1}{\mu^2} &= \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2} \\
&= \frac{1}{\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}} \\
\frac{-2\mu}{\sigma^2} &= \frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2} \\
&= \frac{\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2}}{\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}}
\end{aligned}$$

Therefore, $\sigma^2 = \frac{1}{\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}}$ and $\mu = \frac{\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2}}{\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}}$. Hence $p(x) = \mathcal{N}\left(\frac{\frac{\mu_1}{\sigma_1^2} + \frac{\mu_2}{\sigma_2^2}}{\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}}, \frac{1}{\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}}\right)$.

2. Since $p_i(x)$ is Gaussian, this make p(x) non-negative. We must now prove that the total probability equals to 1. We can do the following:

$$\begin{aligned}
\int_{-\infty}^{\infty} p(x) dx &= \int_{-\infty}^{\infty} \frac{1}{2} (p_1(x) + p_2(x)) \\
&= \frac{1}{2} \left(\int_{-\infty}^{\infty} p_1(x) dx + \int_{-\infty}^{\infty} p_2(x) dx \right) \\
&= \frac{1}{2} (1 + 1) \\
&= 1
\end{aligned}$$

Therefore, p(x) is a valid density. However, p(x) is not a single Gaussian since it has 2 equal weighted components. We can further say

$$p(x) = \frac{1}{2}[(\mathcal{N}(\mu_1, \sigma_1^2)) + (\mathcal{N}(\mu_2, \sigma_2^2))].$$

3. Consider $X = X_1 + X_2$ where $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$. We can calculate the mean of X with the following

$$\mathbb{E}[X] = \mathbb{E}[X_1] + \mathbb{E}[X_2] = \mu_1 + \mu_2$$

Then we can calculate the variance

$$\text{var}(X) = \text{var}(X_1) + \text{var}(X_2) = \sigma_1^2 + \sigma_2^2$$

Therefore, $X \sim \mathcal{N}(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$

4. Let $X = Z^2$ and $Z \sim \mathcal{N}(0, 1)$. Remember that a square can never be 0. Therefore, the support for X is $[0, \infty]$. Since X is not a one-to-one transformation, we will use the CDF method. Recall the CDF
- $$F_x(x) = p(X \leq x) = 0.$$

For $Z^2 \leq X$, we can say that this is equivalent to $-\sqrt{x} \leq Z \leq \sqrt{x}$

$$\begin{aligned} P(Z^2 \leq X) &= P(-\sqrt{x} \leq Z \leq \sqrt{x}) \\ &= \Phi(\sqrt{x}) - \Phi(-\sqrt{x}) \\ &= 2\Phi(\sqrt{x}) - 1 \end{aligned}$$

Then we need to differentiate

$$\begin{aligned} F_x(X) &= \frac{d}{dx}(2\Phi(\sqrt{x}) - 1) \\ &= 2\Phi(\sqrt{x}) \frac{d}{dx}(\sqrt{x}) \end{aligned}$$

Remember that $\Phi(t) = \frac{1}{\sqrt{2\pi}}e^{-\frac{t^2}{2}}$, so we have

$$F_x(X) = 2\left(\frac{1}{\sqrt{2\pi}}e^{-\frac{x^2}{2}}\right)\left(\frac{1}{2\sqrt{x}}\right) = \frac{1}{\sqrt{2\pi x}}e^{-\frac{x}{2}} \text{ as the density of } X.$$

Problem 3

1.

$$\begin{aligned}
\log(L(\theta)) &= \log(\prod_{i=1}^n p_{\theta}(x_i)) \\
&= \log\left(\frac{\prod_{i=1}^n \exp \theta_{x_i}}{(\sum_{j=1}^K \exp(\theta_j))^n}\right) \\
&= \log\left(\frac{\exp(\sum_{k=1}^K K(n_k \theta_k))}{(\sum_{j=1}^K K \exp(\theta_j))^n}\right) \\
&= \log(\exp(\sum_{k=1}^K K n_k \theta_k)) - \log((\sum_{j=1}^K \exp(\theta_j))^n) \\
&= \sum_{k=1}^K n_k \theta_k - n \log(\sum_{j=1}^K \exp(\theta_j))
\end{aligned}$$

$$\begin{aligned}
\nabla_{\theta} l(\theta) &= \nabla_{\theta} \sum_{k=1}^K (n_k \theta_k) - \nabla_{\theta} n \log(\sum_{j=1}^K (\exp(\theta_j))) \\
&= n_r - n \frac{\exp(\theta_r)}{\sum_{j=1}^K (\exp(\theta_j))} \\
&= n_r - n \frac{\exp(\theta_r)}{\sum_{j=1}^K (\exp(\theta_j))} \\
&= n_r - n * \text{softmax}(r)
\end{aligned}$$

2.

With this, we can set the gradient to 0 for maximum likelihood.

$$\begin{aligned}
n_r - n * \text{softmax}(r) &= 0 \\
n_r &= n * \text{softmax}(r) \\
\frac{n_r}{n} &= \text{softmax}(r)
\end{aligned}$$

We can see here that the maximum likelihood is equivalent to the empirical distribution. Therefore, θ is not unique. To further prove this, let

$$Z = \sum_{j=1}^K (\exp(\theta_j)).$$

$$\begin{aligned}
\frac{\exp(\theta_r)}{\sum_{j=1}^K (\exp(\theta_j))} &= \frac{n_r}{n} \\
\frac{\exp(\theta_r)}{Z} &= \frac{n_r}{n} \\
\exp(\theta_r) &= \frac{Z}{n} n_r
\end{aligned}$$

Taking the log of this function will give us $\theta_r = \log(n_r) + C$. Since C is a constant and can be any number, it tells us that θ_r is not unique. However, the induced distribution is unique with the satisfaction of $p_{\theta}(k) = \frac{n_k}{n}$.

3. For the following problems, let $\exp(10000) = \infty$ and $\exp(20000) = \infty$ due to the extremely large value it actually evaluates to for simplicity sake. Let $\exp(-10000) = 0$ and $\exp(-20000) = 0$ for simplicity sake too. We can use this to evaluate the following.

1. $\frac{\exp(10000)}{\exp(20000) + \exp(10000)} = \frac{\infty}{\infty + \infty} = \infty$ therefore, this is a NaN.
2. $\frac{\exp(-20000)}{\exp(-10000) + \exp(-20000)} = \frac{0}{0+0}$ and 0 cannot be a denominator making this number a NaN.
3. $\frac{\exp(-10000)}{\exp(-10000) + \exp(0)} = \frac{0}{0+1} = 0$ this is numerically stable however due to the level of precision needed to evaluate this there exists a loss of significance.
4. A stable version of a softmax is $p_{\theta}(k) = \frac{\exp(\theta_k - M)}{\sum_{j=1}^K (\exp(\theta_j - M))}$ where $M = \max_j(\theta_j)$. Since M is a constant, we know that the softmax function is still unchanged. Taking the logarithm of this function we get

$$m + \log\left(\sum_{j=1}^K (\exp(\theta_j - m))\right)$$

Subbing this back into the original equation

$$l(\theta) = \sum_{k=1}^K (n_k \theta_k) - n(m + \log\left(\sum_{j=1}^K (\exp(\theta_j - m))\right))$$

From this, we know $\theta_j - m \leq 0$ therefore implying $\exp(\theta_j - m) \leq 1$ therefore overflow cannot occur.

Problem 4

Neural Energy Models

We want to fit a dataset $\mathcal{D} = \{x_i\}_{i=1}^n$ with an energy model of form

$$p_{\theta}(x) = \frac{\exp(f_{\theta}(x))}{Z_{\theta}}, \quad Z_{\theta} = \int \exp(f_{\theta}(x)) dx,$$

where $f_{\theta}(x)$ is an energy function, and Z_{θ} is the partition function. To make the ensure that the integration is finite, we may assume that

$$f_{\theta}(x) = n n_{\theta}(x) - \frac{\|x - \mu\|^2}{2\sigma^2},$$

where $nn_\theta(x)$ is a neural network, such as MLP, with trainable weights θ , μ, σ can be either a fixed hyperparameter (e.g., $\mu = 0, \sigma = 1$), or trainable.

Maximum Likelihood Estimator (MLE)

We will try maximum likelihood estimator (MLE). The log-likelihood function is

$$\ell(\theta; \mathcal{D}) = \mathbb{E}_{x \sim \mathcal{D}}[\log p_\theta(x)] = \mathbb{E}_{x \sim \mathcal{D}}[f_\theta(x)] - \log Z_\theta.$$

Taking the gradient:

$$\nabla_\theta \ell(\theta; \mathcal{D}) = \mathbb{E}_{x \sim \mathcal{D}}[\nabla_\theta f_\theta(x)] - \mathbb{E}_{x \sim p_\theta}[\nabla_\theta f_\theta(x)].$$

Assume we draw a sample $\hat{\mathcal{D}}_\theta \stackrel{\text{def}}{=} \{\hat{x}_i\}_{i=1}^n$ from p_θ . The gradient can be estimated via

$$\nabla_\theta \ell(\theta; \mathcal{D}) \approx \mathbb{E}_{x \sim \mathcal{D}}[\nabla_\theta f_\theta(x)] - \mathbb{E}_{\hat{x} \sim \hat{\mathcal{D}}_\theta}[\nabla_\theta f_\theta(\hat{x})].$$

In practice, however, we need to find approximate ways to draw $\hat{\mathcal{D}}_\theta$. In the toy example below, we will try a brute force way where we partition the space on grids and draw sample on the grids. This only works for low dimensional cases.

Langevin dynamics is an approximate method for sampling from an energy model. Starting from an initialization x_0 , it performs iterative updates:

$$x^{t+1} \leftarrow x^t + \frac{\epsilon}{2} \nabla_x \log f_\theta(x^t) + \sqrt{\epsilon} \xi^t, \quad \xi^t \stackrel{iid}{\sim} \mathcal{N}(0, I),$$

where ϵ is a small step size. It is expected that x^t approximately follows p_θ when t is large and ϵ is small.

```
In [ ]: #@title Generate 2D GMM Dataset
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from collections import namedtuple

DataInfo = namedtuple('data_info', ['centers', 'cluster_idx'])
def sample_circular_gmm(batch_size, num_modes=4, radius=2.0, std=0.1):
    angles = torch.linspace(0, 2 * torch.pi, num_modes+1)[:num_modes] + torch.rand(1)
    centers = torch.stack([radius * torch.cos(angles), radius * torch.sin(angles)])
    cluster_idx = torch.randint(0, num_modes, (batch_size,))
    selected_centers = centers[cluster_idx]
    batch_data = selected_centers + std * torch.randn(batch_size, 2)

    data_info = DataInfo(centers=centers, cluster_idx=cluster_idx)
```

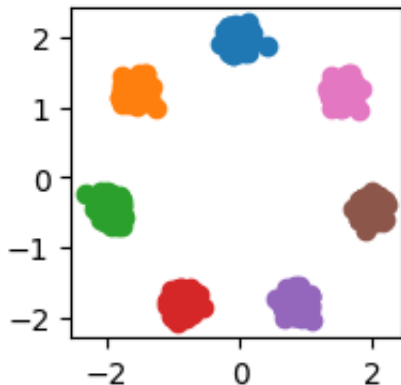
```

    return batch_data, data_info

# Visualize the data
data_points, data_info = sample_circular_gmm(1000, num_modes=7)

plt.figure(figsize=(2, 2))
for i in range(len(data_info.centers)):
    idx = (data_info.cluster_idx == i)
    plt.scatter(data_points[idx, 0], data_points[idx, 1])

```



```

In [ ]: #@title Energy Model, implemented brute force sampling.
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import math

class EnergyModel(nn.Module):
    def __init__(self, model, input_dim):
        super(EnergyModel, self).__init__()
        self.energy = model # the negative energy
        self.temperature = 1.0

        # add a learnable gaussian prior term to the energy function
        self.mean_prior = nn.Parameter(torch.zeros(input_dim))
        self.inv_std_prior = nn.Parameter(torch.ones(input_dim))
        self.energy_grid_cache = None

    def neg_log_gaussian_prior(self, x):
        # (x-mu)^2/(2sigma^2)
        return torch.sum((self.inv_std_prior[None,:]*(x-self.mean_prior

    def forward(self, x):
        # total energy = (neural energy + .5*(inv_std*(x-mu))*2)/temp
        return (self.energy(x) - self.neg_log_gaussian_prior(x))/self.

    # calculating logZ by grid sum
    def logZ_2D_grid(self, x_range=torch.linspace(-5, 5, 100), y_range=
        X, Y, Z = self.update_energy_2D_grid(x_range=x_range, y_range=

```

```

        logZ = torch.logsumexp(Z.flatten(), dim=0) # correct upto a co
        return logZ

# Evaluate the energy model on the grid, and save it in cache (only
def update_energy_2D_grid(self, x_range=torch.linspace(-5, 5, 100)
    X, Y = torch.meshgrid(x_range, y_range, indexing='ij')
    input_grid = torch.stack([X.flatten(), Y.flatten()], dim=1) #
    Z = self(input_grid).reshape(X.shape)
    self.energy_grid_cache = [X,Y,Z]
    return X, Y, Z

# get cached energy
def retrieve_energy_2D_grid(self, x_range=torch.linspace(-5, 5, 100
    if self.energy_grid_cache is None:
        X,Y,Z = self.update_energy_2D_grid()
    else:
        X,Y,Z = self.energy_grid_cache
    return X, Y, Z

# plot energy contour
@torch.no_grad()
def plot_energy_2D_contour(self, x_range=torch.linspace(-5, 5, 100
    X, Y, Z = self.retrieve_energy_2D_grid(x_range, y_range)
    plt.contour(X.detach().cpu(), Y.detach().cpu(), Z.detach().cpu
    plt.contourf(X.detach().cpu(), Y.detach().cpu(), Z.detach().cp

# sampling from the energy model on the grid (as an approximation)
@torch.no_grad()
def sampling_2D_grid(self, num_samples, x_range=torch.linspace(-5,
    X, Y, Z = self.retrieve_energy_2D_grid(x_range=x_range, y_rang
    prob = torch.softmax(Z.flatten(), dim=0)
    index = torch.multinomial(prob, num_samples=num_samples)
    return torch.stack([X.flatten()[index], Y.flatten()[index]], d

mlp = nn.Sequential(
    nn.Linear(2, 64),
    nn.ReLU(),
    nn.Linear(64, 64),
    nn.ReLU(),
    nn.Linear(64, 1))
model = EnergyModel(mlp, input_dim = 2)

logZ = model.logZ_2D_grid()

print(f'logZ, calculated by grid estimation:{logZ: .2f}')

plt.figure(figsize=(5,5))

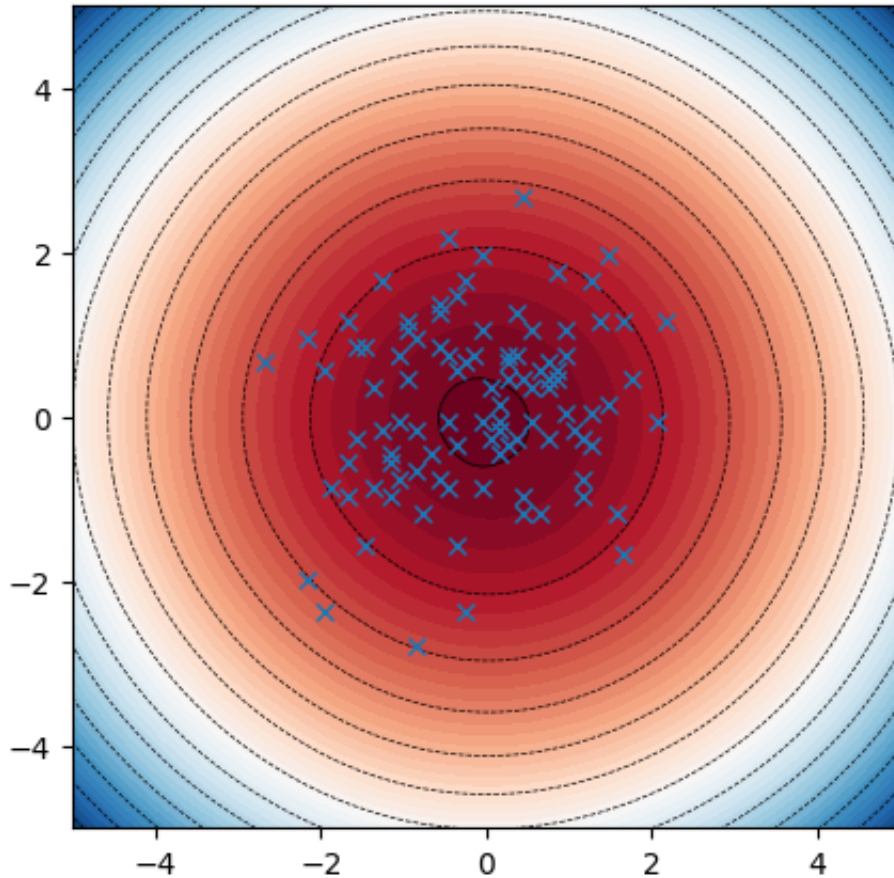
model.plot_energy_2D_contour()

x_grid_sample = model.sampling_2D_grid(100)
plt.plot(x_grid_sample[:,0], x_grid_sample[:,1], 'x')

```

```
plt.show()
```

logZ, calculated by grid estimation: 6.61



```
In [ ]: #@title Langevin sampler (Please complete)

def langevin_sampler(model, x0, num_steps=100, step_size=0.01, record
    x = x0.detach().requires_grad_(True) # Ensure x requires grad for
    traj = [x.detach()]
    for iter in range(num_steps):
        grad_energy = torch.autograd.grad(model(x).sum(), x)[0]
        #####
        # write the update of Langevin sampler here
        # x = x + ....
        #####
        x = x + 0.5 * step_size * grad_energy + torch.sqrt(torch.tensor(

        if record_traj: traj.append(x.detach())

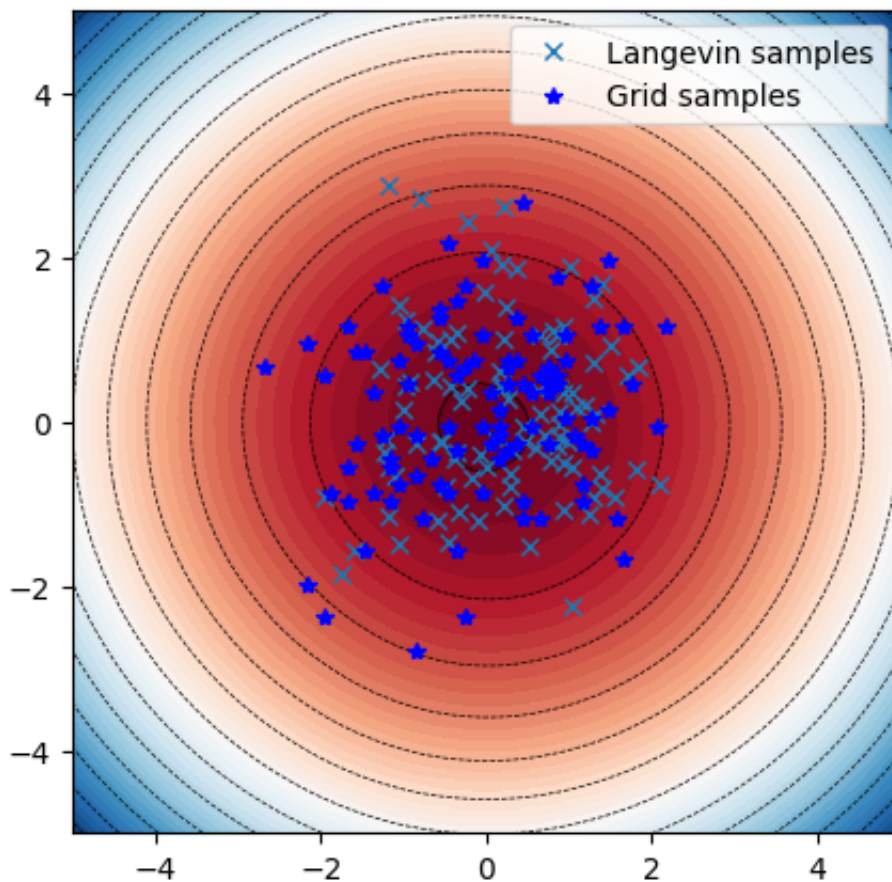
    if record_traj:
        return x.detach(), torch.stack(traj)
    else:
        return x.detach()

x0 = torch.randn(100,2)
x_langevin = langevin_sampler(model, x0, num_steps=100, step_size=0.0
plt.figure(figsize=(5,5))
```

```

model.plot_energy_2D_contour()
plt.plot(x_langevin[:,0], x_langevin[:,1], 'x', label='Langevin sample')
plt.plot(x_grid_sample[:,0], x_grid_sample[:,1], 'b*', label='Grid sample')
plt.legend()
plt.show()

```



Both methods produce samples concentrated near the origin, which corresponds to the highest-density region of the target distribution. The overall shape of the samples generated by both methods is similar, indicating that Langevin dynamics is able to successfully approximate the distribution produced by the brute-force grid sampler. The Langevin samples also explore regions farther from the origin, including lower-density areas. This behavior is due to the noise term in the Langevin update, as well as the finite step size used during sampling. Overall, these results demonstrate that Langevin dynamics can effectively approximate the target distribution while requiring only gradient evaluations rather than an exhaustive grid search.

```

In [ ]: #@title MLE Training (please complete)

from IPython.display import clear_output

# data
input_dim = 2
data, data_info = sample_circular_gmm(1000, num_modes=7)

```

```

# model
mlp = nn.Sequential(
    nn.Linear(input_dim, 64),
    nn.ReLU(),
    nn.Linear(64, 64),
    nn.ReLU(),
    nn.Linear(64, 1))
energy_model = EnergyModel(mlp, input_dim)

# set up optimizer
batch_size = 1000
optimizer = optim.Adam(energy_model.parameters(), lr=1e-3)

# training loop
loss_curve = []
for iter in range(1000):

    optimizer.zero_grad()

    data_batch = data[torch.randperm(len(data))[:batch_size]]

    #####
    # calculate the negative log likelihood on data_batch
    #loss = ...
    #####
    loss = -energy_model(data_batch).mean() + energy_model.logZ_2D_grid(

    loss.backward()
    optimizer.step()
    loss_curve.append(loss.item())

# plot
if iter % 100 == 0:
    clear_output(wait=True)
    KK = 10; kk=1
    plt.figure(figsize=(3*KK,3))

    plt.subplot(1, KK, kk); kk+=1
    plt.plot(loss_curve, label='Training Loss')
    plt.title('Negative Log-Likelihood')
    plt.legend()

    plt.subplot(1, KK, kk); kk+=1
    energy_model.plot_energy_2D_contour()
    plt.scatter(data_batch[:, 0], data_batch[:, 1], alpha=0.5, label='')
    plt.title('Data vs. Energy')

    plt.subplot(1, KK, kk); kk+=1
    x0 = torch.randn((100,2))
    energy_model.temperature = 1.0
    data_langevin, traj = langevin_sampler(energy_model, x0, num_step

```

```

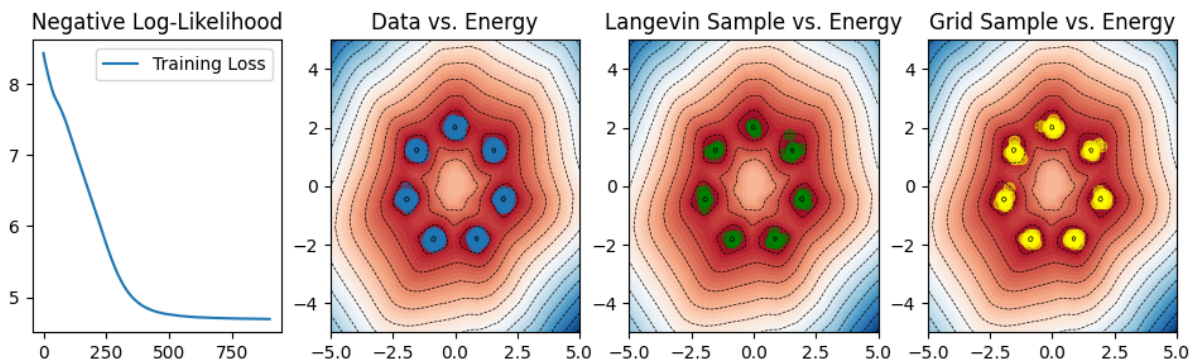
energy_model.plot_energy_2D_contour()
plt.scatter(data_langevin[:, 0], data_langevin[:, 1], alpha=0.5, l
plt.title('Langevin Sample vs. Energy')

plt.subplot(1, KK, kk); kk+=1
energy_model.plot_energy_2D_contour()
data_grid = energy_model.sampling_2D_grid(100)
plt.scatter(data_grid[:, 0], data_grid[:, 1], alpha=0.5, label='re
plt.title('Grid Sample vs. Energy')

plt.show()
plt.suptitle(f'iter={iter}')

```

<Figure size 640x480 with 0 Axes>



<Figure size 640x480 with 0 Axes>

PART 2

Problem 2

```

In [ ]: import math
import numpy as np
import matplotlib.pyplot as plt
from typing import Tuple, Dict

```

Running cells with 'Python 3.9.6' requires the ipykernel package.

Or install 'ipykernel' using the command: '/usr/bin/python3 -m pip install ipykernel -U --user --force-reinstall'

Objective function and analytic gradient

We will use a **two-dimensional non-convex test function**, formed by combining a quadratic bowl with two Gaussian wells. The function is defined as:

$$f(x, y) = -2 \exp\left(-\frac{(x-1)^2 + y^2}{0.2}\right) - 3 \exp\left(-\frac{(x+1)^2 + y^2}{0.2}\right) + x^2 + y^2.$$

This creates two Gaussian “wells” centered at $(1, 0)$ and $(-1, 0)$ (the left one being deeper), plus a quadratic bowl centered at the origin.

Let

$$e_1 = \exp\left(-\frac{(x-1)^2+y^2}{0.2}\right), \quad e_2 = \exp\left(-\frac{(x+1)^2+y^2}{0.2}\right).$$

The gradient is:

$$\nabla f(x, y) = \begin{bmatrix} 2x + 20(x-1)e_1 + 30(x+1)e_2 \\ 2y + 20ye_1 + 30ye_2 \end{bmatrix}.$$

```
In [ ]: import math
import numpy as np
import matplotlib.pyplot as plt

# -----
# Define objective function f(x,y)
# -----
def f(x: float, y: float) -> float:
    """
    Objective: quadratic bowl + two Gaussian wells.
    """
    # TODO: fill in the exponential terms e1 and e2
    e1 = math.exp(-((x - 1.0) ** 2 + y ** 2) / 0.20) # well centered at (1,0)
    e2 = math.exp(-((x + 1.0) ** 2 + y ** 2) / 0.20) # well centered at (-1,0)

    return -2.0 * e1 - 3.0 * e2 + x * x + y * y

# -----
# Gradient ∇f(x,y)
# -----
def grad(x: float, y: float) -> np.ndarray:
    """
    Analytic gradient of f(x,y).
    """
    # TODO: fill in the exponential terms e1 and e2
    e1 = math.exp(-((x - 1.0) ** 2 + y ** 2) / 0.20)
    e2 = math.exp(-((x + 1.0) ** 2 + y ** 2) / 0.20)

    # TODO: fill in gx = ∂f/∂x, gy = ∂f/∂y
    gx = 2 * x + 20.0 * (x - 1.0) * e1 + 30 * (x + 1.0) * e2
    gy = 2 * y + 20 * y * e1 + 30 * y * e2

    return np.array([gx, gy], dtype=float)

# -----
```

```

# Plot contour of f(x,y)
# -----
xlim = (-2.0, 2.0)
ylim = (-2.0, 2.0)
res = 400

xs = np.linspace(*xlim, res)
ys = np.linspace(*ylim, res)
XX, YY = np.meshgrid(xs, ys)

ZZ = np.vectorize(f)(XX, YY)

fig, ax = plt.subplots(figsize=(9, 5), dpi=150)

cf = ax.contourf(XX, YY, ZZ, levels=40, alpha=0.85)
ax.contour(XX, YY, ZZ, levels=40, linewidths=0.6)

fig.colorbar(cf, ax=ax, label='f(x, y)')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Contour map of f(x, y)')
ax.set_xlim(xlim)
ax.set_ylim(ylim)
ax.grid(True, alpha=0.3)

plt.show()

```

Optimizers

We will implement and compare several optimization algorithms. Each algorithm updates the parameters (x_t, y_t) using the gradient $\nabla f(x_t, y_t)$.

1. Gradient Descent (GD)

The simplest method performs a step in the **negative gradient direction**:

$$(x_{t+1}, y_{t+1}) = (x_t, y_t) - \eta \nabla f(x_t, y_t),$$

where $\eta > 0$ is the learning rate.

2. Momentum

Momentum introduces a velocity vector v_t that accumulates past gradients:

$$\begin{aligned} v_{t+1} &= \mu v_t - \eta \nabla f(x_t, y_t), \\ (x_{t+1}, y_{t+1}) &= (x_t, y_t) + v_{t+1}, \end{aligned}$$

where $0 \leq \mu < 1$ is the momentum parameter.

3. Signed Gradient Descent (SignGD)

Instead of using the gradient magnitude, we only use the **sign** of each component:

$$(x_{t+1}, y_{t+1}) = (x_t, y_t) - \eta \text{sign}(\nabla f(x_t, y_t)).$$

This results in uniform-sized steps in each coordinate direction.

4. Soft-Signed Gradient Descent

To smooth the discontinuity of the sign function, we apply a **softsign** transform with parameter $\tau > 0$:

$$\text{softsign}(z) = \frac{z/\tau}{1 + |z|/\tau}.$$

The update becomes

$$(x_{t+1}, y_{t+1}) = (x_t, y_t) - \eta \text{softsign}(\nabla f(x_t, y_t)).$$

5. RMSProp

RMSProp normalizes the gradient by a running average of its squared values:

$$\begin{aligned} v_{t+1} &= \beta v_t + (1 - \beta) (\nabla f(x_t, y_t))^{\odot 2}, \\ (x_{t+1}, y_{t+1}) &= (x_t, y_t) - \eta \frac{\nabla f(x_t, y_t)}{\sqrt{v_{t+1}} + \varepsilon}, \end{aligned}$$

where $\odot$ denotes elementwise square.

6. Adam

Adam combines **momentum** and **adaptive scaling** (like RMSProp). It maintains first- and second-moment estimates:

$$\begin{aligned} m_{t+1} &= \beta_1 m_t + (1 - \beta_1) \nabla f(x_t, y_t), \\ v_{t+1} &= \beta_2 v_t + (1 - \beta_2) (\nabla f(x_t, y_t))^{\odot 2}, \\ \hat{m}_{t+1} &= \frac{m_{t+1}}{1 - \beta_1^{t+1}}, \quad \hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}, \\ (x_{t+1}, y_{t+1}) &= (x_t, y_t) - \eta \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1}} + \varepsilon}. \end{aligned}$$

```

In [ ]: # -----
# Optimizers (fill in the TODO parts)
# -----

def run_gd(x0: float, y0: float, steps: int, lr: float):
    x, y = x0, y0
    traj = [(x, y)]
    losses = [f(x, y)]
    for _ in range(steps):
        g = grad(x, y)
        # TODO: update x, y using gradient descent
        x = x - lr * g[0]
        y = y - lr * g[1]
        traj.append((x, y))
        losses.append(f(x, y))
    return np.array(traj), np.array(losses)

def run_momentum(x0: float, y0: float, steps: int, lr: float, mu: float):
    x, y = x0, y0
    v = np.zeros(2) # velocity
    traj = [(x, y)]
    losses = [f(x, y)]
    for _ in range(steps):
        g = grad(x, y)
        # TODO: update velocity v and parameters x, y
        v = mu * v - lr * g
        x = x + v[0]
        y = y + v[1]
        traj.append((x, y))
        losses.append(f(x, y))
    return np.array(traj), np.array(losses)

def run_sign_gd(x0: float, y0: float, steps: int, lr: float):
    """Elementwise sign(grad) step (a.k.a. signSGD)."""
    x, y = x0, y0
    traj = [(x, y)]
    losses = [f(x, y)]
    for _ in range(steps):
        g = grad(x, y)
        # TODO: update x, y using only the sign of g
        step = np.sign(g)
        x = x - lr * step[0]
        y = y - lr * step[1]
        traj.append((x, y))
        losses.append(f(x, y))
    return np.array(traj), np.array(losses)

def run_softsign_gd(x0: float, y0: float, steps: int, lr: float, tau:

```

```

"""
Soft-signed step: elementwise softsign(g/tau) = (g/tau) / (1 + |g|)
Saturates like sign but remains smooth around 0.
"""

x, y = x0, y0
traj = [(x, y)]
losses = [f(x, y)]
for _ in range(steps):
    g = grad(x, y)
    # TODO: compute softsign of g, then update x, y
    z = (g / tau) / (1 + np.abs(g) / tau)
    soft = np.sign(z)
    step = lr * soft
    x = x - step[0]
    y = y - step[1]
    traj.append((x, y))
    losses.append(f(x, y))
return np.array(traj), np.array(losses)

def run_rmsprop(x0: float, y0: float, steps: int, lr: float, beta: float):
    x, y = x0, y0
    v = np.zeros(2) # running average of squared gradients
    traj = [(x, y)]
    losses = [f(x, y)]
    for _ in range(steps):
        g = grad(x, y)
        # TODO: update running average v and parameters x, y
        v = beta * v + (1 - beta) * g * g
        step = lr * g / (np.sqrt(v) + eps)
        x = x - step[0]
        y = y - step[1]
        traj.append((x, y))
        losses.append(f(x, y))
    return np.array(traj), np.array(losses)

def run_adam(x0: float, y0: float, steps: int, lr: float, b1: float = 0.9, b2: float = 0.999):
    x, y = x0, y0
    m = np.zeros(2) # first moment (mean of gradients)
    v = np.zeros(2) # second moment (mean of squared gradients)
    traj = [(x, y)]
    losses = [f(x, y)]
    for t in range(1, steps + 1):
        g = grad(x, y)
        # TODO: update m, v, compute bias-corrected mhat, vhat, then u
        m = b1 * m + (1 - b1) * g
        v = b2 * v + (1 - b2) * g * g
        mhat = m / (1 - b1 ** t)
        vhat = v / (1 - b2 ** t)
        step = lr * mhat / (np.sqrt(vhat) + eps)
        x = x - step[0]

```

```

        y = y - step[1]
        traj.append((x, y))
        losses.append(f(x, y))
    return np.array(traj), np.array(losses)

```

```

In [ ]: # -----
# Optimizers
# -----

def run_all(x0: float, y0: float, steps: int, HYPERS):
    results = {}
    results["GD"] = run_gd(x0, y0, steps, **HYPERS["GD"])
    results["Momentum"] = run_momentum(x0, y0, steps, **HYPERS["Momen
    results["SignedGD"] = run_sign_gd(x0, y0, steps, **HYPERS["Signed
    results["SoftSignGD"] = run_softsign_gd(x0, y0, steps, **HYPERS["So
    results["RMSProp"] = run_rmsprop(x0, y0, steps, **HYPERS["RMSPro
    results["Adam"] = run_adam(x0, y0, steps, **HYPERS["Adam"])
    return results

def draw(results: Dict[str, tuple], xlim=(-2, 2), ylim=(-2, 2)):
    xs = np.linspace(xlim[0], xlim[1], 300)
    ys = np.linspace(ylim[0], ylim[1], 300)
    XX, YY = np.meshgrid(xs, ys)
    ZZ = np.vectorize(f)(XX, YY)

    fig, ax = plt.subplots(figsize=(8, 6), dpi=300)
    cf = ax.contourf(XX, YY, ZZ, levels=30, alpha=0.8)
    ax.set_facecolor("white")
    ax.contour(XX, YY, ZZ, levels=30, linewidths=0.5)

    styles = {
        "GD": "-.",
        "Momentum": (0, (1, 1)),
        "SignedGD": "-",
        "SoftSignGD": (0, (1, 1)),
        "RMSProp": (0, (3, 1, 1, 1)),
        "Adam": (0, (5, 1, 1, 1)),
    }

    # Plot each path
    for name, (traj, losses) in results.items():
        ax.plot(traj[:, 0], traj[:, 1], linestyle=styles.get(name, "-")
        ax.plot(traj[0, 0], traj[0, 1], marker="o", markersize=4)
        ax.plot(traj[-1, 0], traj[-1, 1], marker="x", markersize=6)

    ax.set_xlim(xlim)
    ax.set_ylim(ylim)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_title("Optimization trajectories on a 2D landscape")
    ax.legend(loc="upper left", frameon=True)

```

```

out_path = "./optimizer_trajectories.png"
fig.tight_layout()
fig.savefig(out_path, dpi=300)
plt.show()
return out_path

def print_final_points_and_losses(results: Dict[str, tuple], start=None,
    if steps is None:
        any_name = next(iter(results))
        steps = len(results[any_name][1]) - 1

    header = f"Final results after {steps} steps"
    if start is not None:
        header += f", start = ({start[0]:.4f}, {start[1]:.4f})"
    print(header)

    for name, (traj, losses) in results.items():
        xT, yT = traj[-1, 0], traj[-1, 1]
        fT = losses[-1]
        print(f"{name:>10s}: f(x_T) = {fT:.6f}, at (x_T, y_T) = ({xT}

    best_name, best_val = min(
        ((n, ls[-1]) for n, (_, ls) in results.items()),
        key=lambda kv: kv[1]
    )
    print(f"\nBest (lowest loss): {best_name} with f(x_T) = {best_val:

```

```

In [ ]: # START: Tuple[float, float] = (-1.2, 1.2)
# START: Tuple[float, float] = (1.2, 1.2)
START: Tuple[float, float] = (0.3, 1.5)

# STEPS = 50
# STEPS = 200
STEPS = 500

HYPERPARAMETERS: Dict[str, dict] = {
    "GD": dict(lr=0.05),
    "Momentum": dict(lr=0.03, mu=0.9),
    "SignedGD": dict(lr=0.03),
    "SoftSignGD": dict(lr=0.10, tau=5.0),
    "RMSProp": dict(lr=0.03, beta=0.9, eps=1e-8),
    "Adam": dict(lr=0.05, b1=0.9, b2=0.9, eps=1e-8),
}

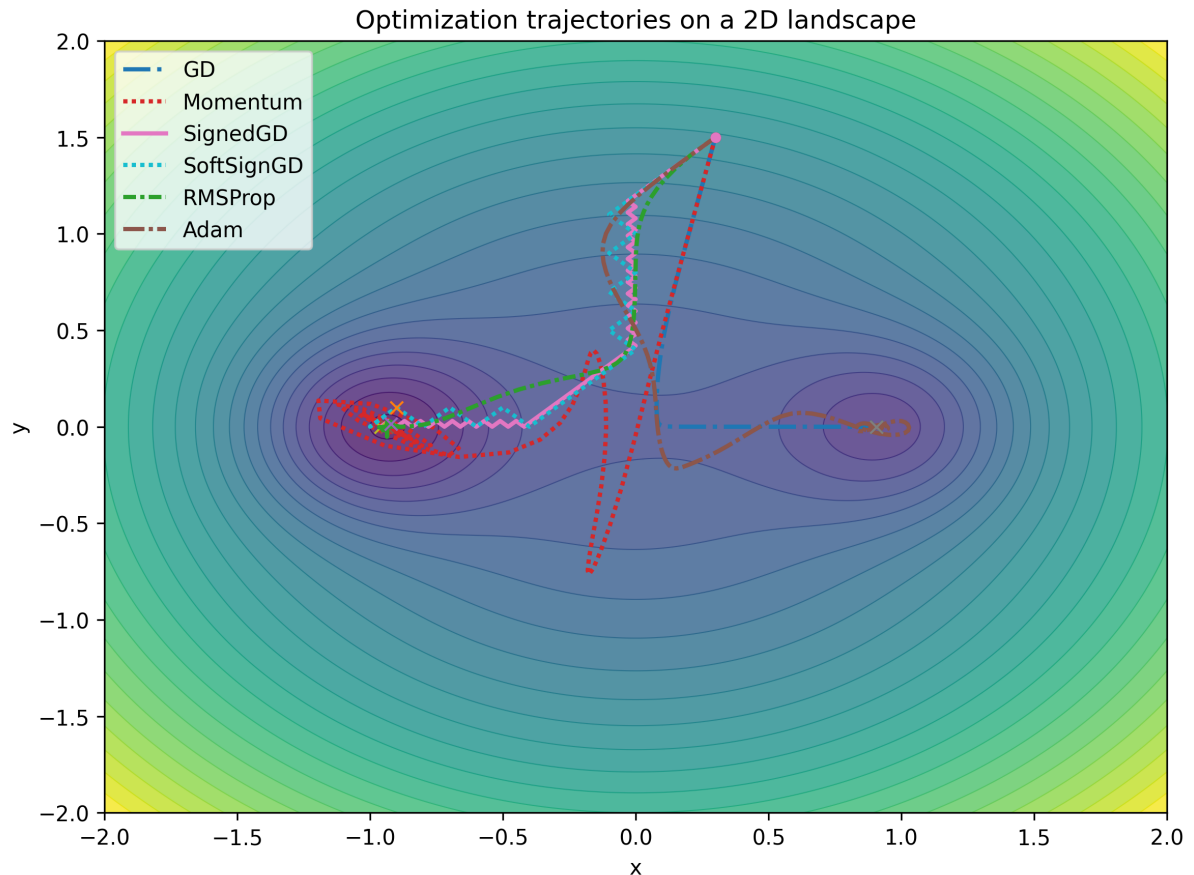
results_ = run_all(*START, STEPS, HYPERPARAMETERS)
print_final_points_and_losses(results_, start=START, steps=STEPS)
png_path = draw(results_)

```

Final results after 500 steps, start = (0.3000, 1.5000)

GD: $f(x_T) = -1.092732$, at $(x_T, y_T) = (0.9053, 0.0000)$
 Momentum: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9363, -0.0000)$
 SignedGD: $f(x_T) = -2.054496$, at $(x_T, y_T) = (-0.9600, -0.0000)$
 SoftSignGD: $f(x_T) = -1.894512$, at $(x_T, y_T) = (-0.9000, 0.1000)$
 RMSProp: $f(x_T) = -2.056043$, at $(x_T, y_T) = (-0.9210, 0.0151)$
 Adam: $f(x_T) = -1.092711$, at $(x_T, y_T) = (0.9062, -0.0011)$

Best (lowest loss): Momentum with $f(x_T) = -2.063090$



```
In [ ]: START: Tuple[float, float] = (-1.2, 1.2)
# START: Tuple[float, float] = (1.2, 1.2)
# START: Tuple[float, float] = (0.3, 1.5)

# STEPS = 50
# STEPS = 200
STEPS = 500

HYPERPS: Dict[str, dict] = {
    "GD": dict(lr=0.05),
    "Momentum": dict(lr=0.03, mu=0.9),
    "SignedGD": dict(lr=0.03),
    "SoftSignGD": dict(lr=0.10, tau=5.0),
    "RMSProp": dict(lr=0.03, beta=0.9, eps=1e-8),
    "Adam": dict(lr=0.05, b1=0.9, b2=0.9, eps=1e-8),
}

results_ = run_all(*START, STEPS, HYPERPS)
```

```
print_final_points_and_losses(results_, start=START, steps=STEPS)
png_path = draw(results_)
```

Final results after 500 steps, start = (-1.2000, 1.2000)

GD: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9363, 0.0000)$

Momentum: $f(x_T) = -1.092732$, at $(x_T, y_T) = (0.9053, -0.0000)$

SignedGD: $f(x_T) = -2.054496$, at $(x_T, y_T) = (-0.9600, -0.0000)$

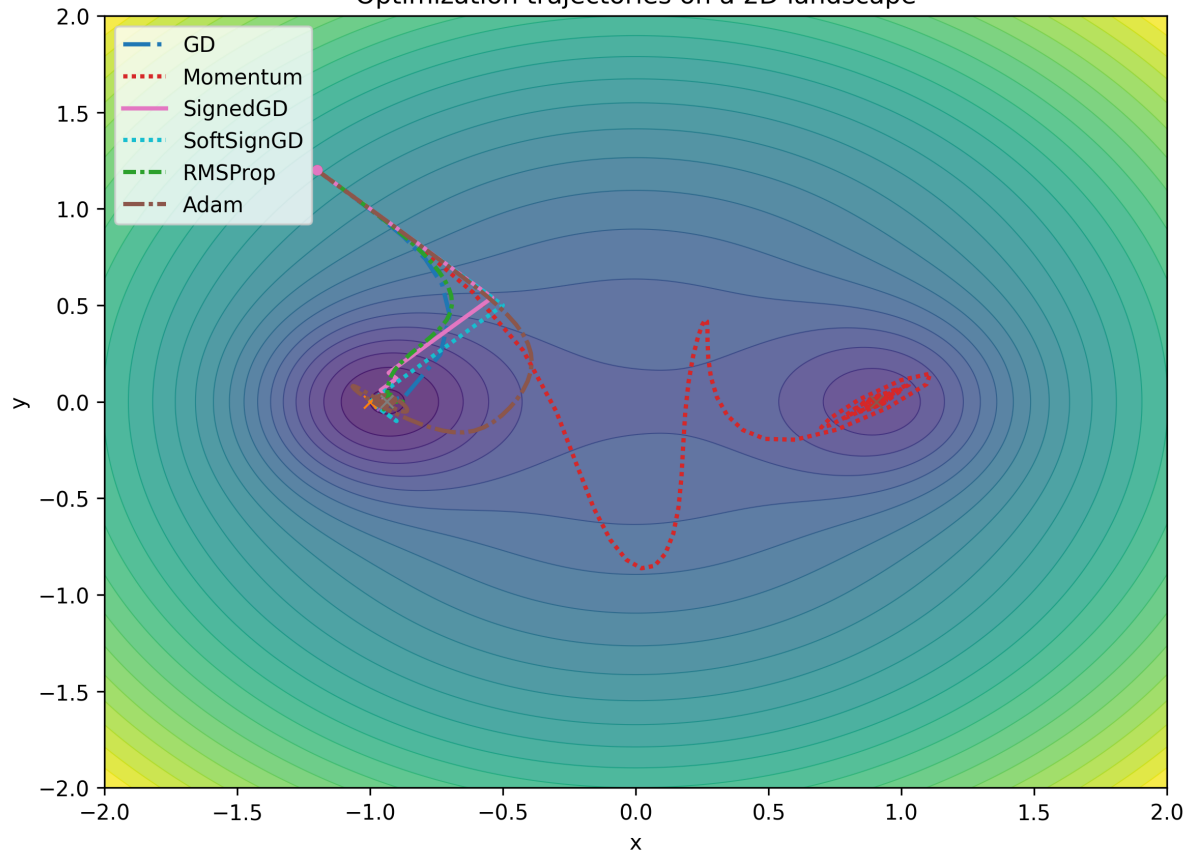
SoftSignGD: $f(x_T) = -2.000000$, at $(x_T, y_T) = (-1.0000, 0.0000)$

RMSProp: $f(x_T) = -2.056295$, at $(x_T, y_T) = (-0.9510, -0.0149)$

Adam: $f(x_T) = -2.063072$, at $(x_T, y_T) = (-0.9373, 0.0003)$

Best (lowest loss): GD with $f(x_T) = -2.063090$

Optimization trajectories on a 2D landscape



```
In [ ]: # START: Tuple[float, float] = (-1.2, 1.2)
START: Tuple[float, float] = (1.2, 1.2)
# START: Tuple[float, float] = (0.3, 1.5)

# STEPS = 50
# STEPS = 200
STEPS = 500

HYPERPARAMS: Dict[str, dict] = {
    "GD": dict(lr=0.05),
    "Momentum": dict(lr=0.03, mu=0.9),
    "SignedGD": dict(lr=0.03),
    "SoftSignGD": dict(lr=0.10, tau=5.0),
    "RMSProp": dict(lr=0.03, beta=0.9, eps=1e-8),
    "Adam": dict(lr=0.05, b1=0.9, b2=0.9, eps=1e-8),
```

```

}

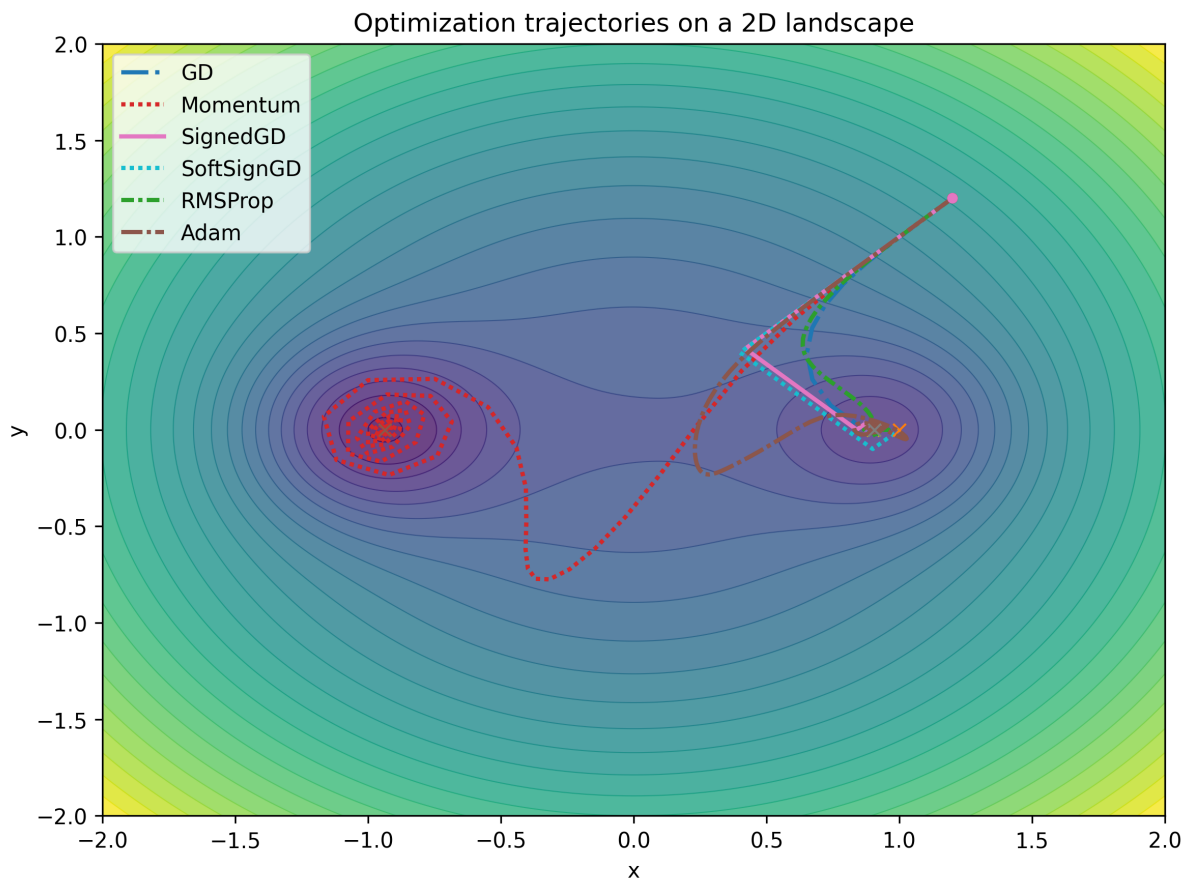
results_ = run_all(*START, STEPS, HYPERS)
print_final_points_and_losses(results_, start=START, steps=STEPS)
png_path = draw(results_)

```

Final results after 500 steps, start = (1.2000, 1.2000)

GD: $f(x_T) = -1.092732$, at $(x_T, y_T) = (0.9053, 0.0000)$
Momentum: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9363, -0.0000)$
SignedGD: $f(x_T) = -1.092459$, at $(x_T, y_T) = (0.9000, -0.0000)$
SoftSignGD: $f(x_T) = -1.000000$, at $(x_T, y_T) = (1.0000, 0.0000)$
RMSProp: $f(x_T) = -1.088058$, at $(x_T, y_T) = (0.8899, 0.0152)$
Adam: $f(x_T) = -1.092661$, at $(x_T, y_T) = (0.9069, -0.0021)$

Best (lowest loss): Momentum with $f(x_T) = -2.063090$



- Each resulting graph is the result of different initializations. From the graphs, it can be seen that Adam and Momentum seem to curve more and overshoot often before settling to a dense part of the graph. RMSProp seems to be the most stable optimizer amongst the optimizers according to the graphs. It seems to go towards the dense area without much curve and overshooting with different initialization values of step size and starting points. For some optimizers like SignedGD seem to lag behind when the step size is small.

```
In [ ]: STARTS: list[Tuple[float, float]] = [(-1.2, 1.2), (1.2, 1.2), (0.3, 1.
```

```

STEPS = [50, 200]

HYPERPARAMETERS = list[Dict[str, dict]] = [
    {
        "GD": dict(lr=0.2),
        "Momentum": dict(lr=0.1, mu=0.8),
        "SignedGD": dict(lr=0.2),
        "SoftSignGD": dict(lr=0.10, tau=1.0),
        "RMSProp": dict(lr=0.05, beta=0.9, eps=1e-8),
        "Adam": dict(lr=0.05, b1=0.9, b2=0.99, eps=1e-8),
    },
    {
        "GD": dict(lr=0.05),
        "Momentum": dict(lr=0.03, mu=0.90),
        "SignedGD": dict(lr=0.05),
        "SoftSignGD": dict(lr=0.05, tau=5.0),
        "RMSProp": dict(lr=0.01, beta=0.95, eps=1e-8),
        "Adam": dict(lr=0.01, b1=0.9, b2=0.999, eps=1e-8),
    },
    {
        "GD": dict(lr=0.05),
        "Momentum": dict(lr=0.01, mu=0.99),
        "SignedGD": dict(lr=0.005),
        "SoftSignGD": dict(lr=0.01, tau=20.0),
        "RMSProp": dict(lr=0.001, beta=0.99, eps=1e-8),
        "Adam": dict(lr=0.001, b1=0.9, b2=0.999, eps=1e-8),
    },
]

for sp in STARTS:
    for step in STEPS:
        print("\n" + "="*80)
        print(f"START={sp}, STEPS={step}")
        print("="*80)

        for hyper_parameter in HYPERPARAMETERS:
            print(f"\nRunning for parameters: {hyper_parameter}\n")

            results_ = run_all(*sp, step, hyper_parameter)
            print_final_points_and_losses(results_, start=sp, steps=step)
            png_path = draw(results_)

        print("\n" + "="*80)

```

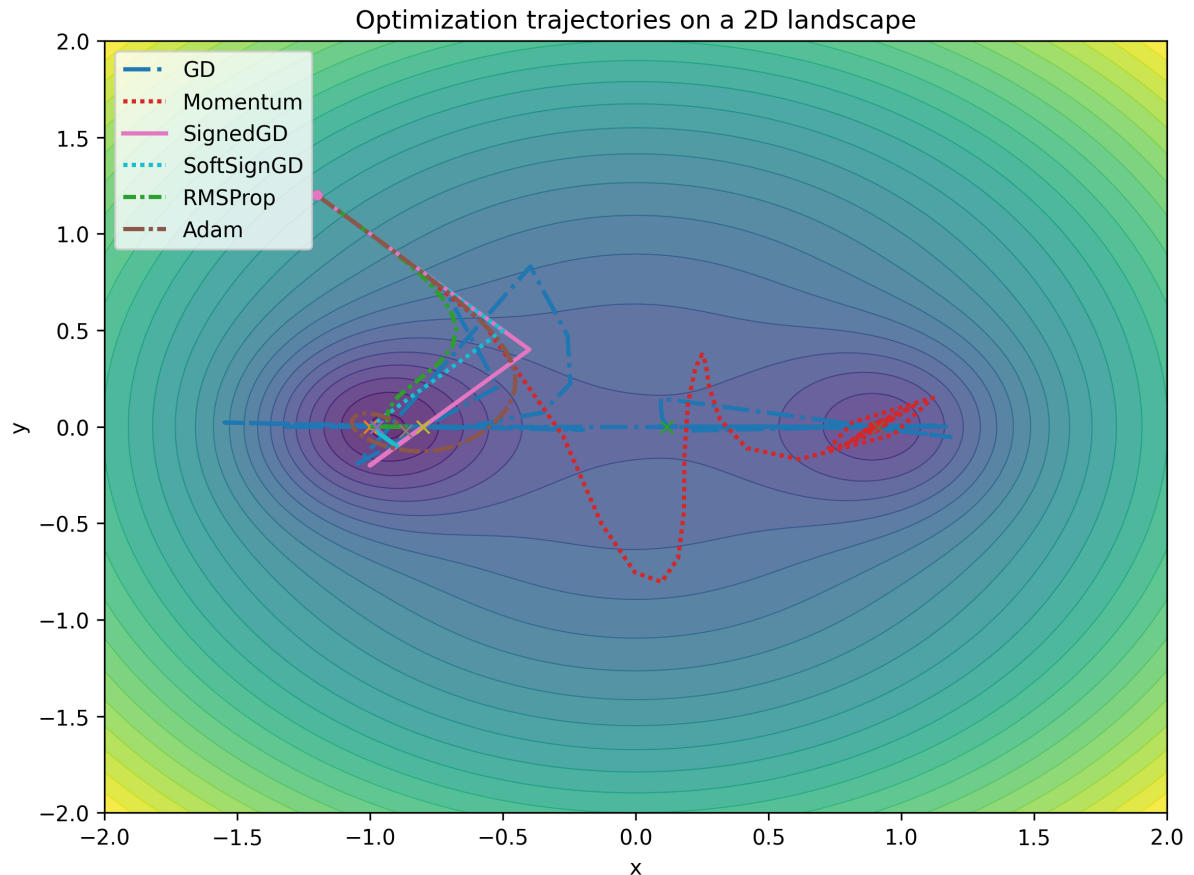
```
=====
START=(-1.2, 1.2), STEPS=50
=====
```

```
Running for parameters: {'GD': {'lr': 0.2}, 'Momentum': {'lr': 0.1, 'mu': 0.8}, 'SignedGD': {'lr': 0.2}, 'SoftSignGD': {'lr': 0.1, 'tau': 1.0}, 'RMSProp': {'lr': 0.05, 'beta': 0.9, 'eps': 1e-08}, 'Adam': {'lr': 0.05, 'b1': 0.9, 'b2': 0.99, 'eps': 1e-08}}
```

```
Final results after 50 steps, start = (-1.2000, 1.2000)
```

```
GD: f(x_T) = -0.032761, at (x_T, y_T) = ( 0.1178, -0.0000)
Momentum: f(x_T) = -1.092138, at (x_T, y_T) = ( 0.9124, -0.0031)
SignedGD: f(x_T) = -1.816192, at (x_T, y_T) = (-0.8000, 0.0000)
SoftSignGD: f(x_T) = -2.000000, at (x_T, y_T) = (-1.0000, 0.0000)
RMSProp: f(x_T) = -2.040991, at (x_T, y_T) = (-0.9742, 0.0000)
Adam: f(x_T) = -1.984521, at (x_T, y_T) = (-0.8655, -0.0216)
```

```
Best (lowest loss): RMSProp with f(x_T) = -2.040991
```

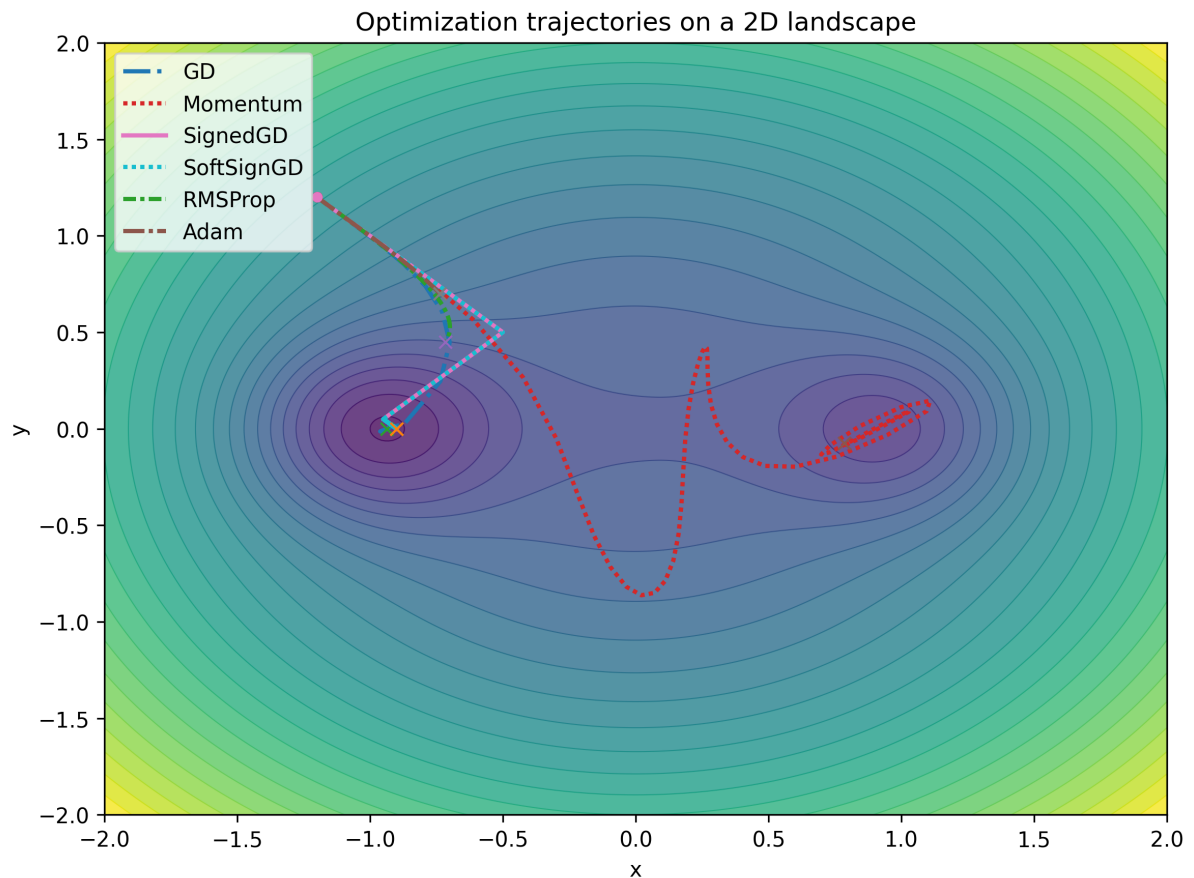


Running for parameters: {'GD': {'lr': 0.05}, 'Momentum': {'lr': 0.03, 'mu': 0.9}, 'SignedGD': {'lr': 0.05}, 'SoftSignGD': {'lr': 0.05, 'tau': 5.0}, 'RMSProp': {'lr': 0.01, 'beta': 0.95, 'eps': 1e-08}, 'Adam': {'lr': 0.01, 'b1': 0.9, 'b2': 0.999, 'eps': 1e-08}}

Final results after 50 steps, start = (-1.2000, 1.2000)

GD: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9363, 0.0000)$
Momentum: $f(x_T) = -0.924728$, at $(x_T, y_T) = (0.7864, -0.0743)$
SignedGD: $f(x_T) = -2.043688$, at $(x_T, y_T) = (-0.9000, -0.0000)$
SoftSignGD: $f(x_T) = -2.043688$, at $(x_T, y_T) = (-0.9000, -0.0000)$
RMSProp: $f(x_T) = -0.015634$, at $(x_T, y_T) = (-0.7158, 0.4493)$
Adam: $f(x_T) = 0.851949$, at $(x_T, y_T) = (-0.7374, 0.7006)$

Best (lowest loss): GD with $f(x_T) = -2.063090$

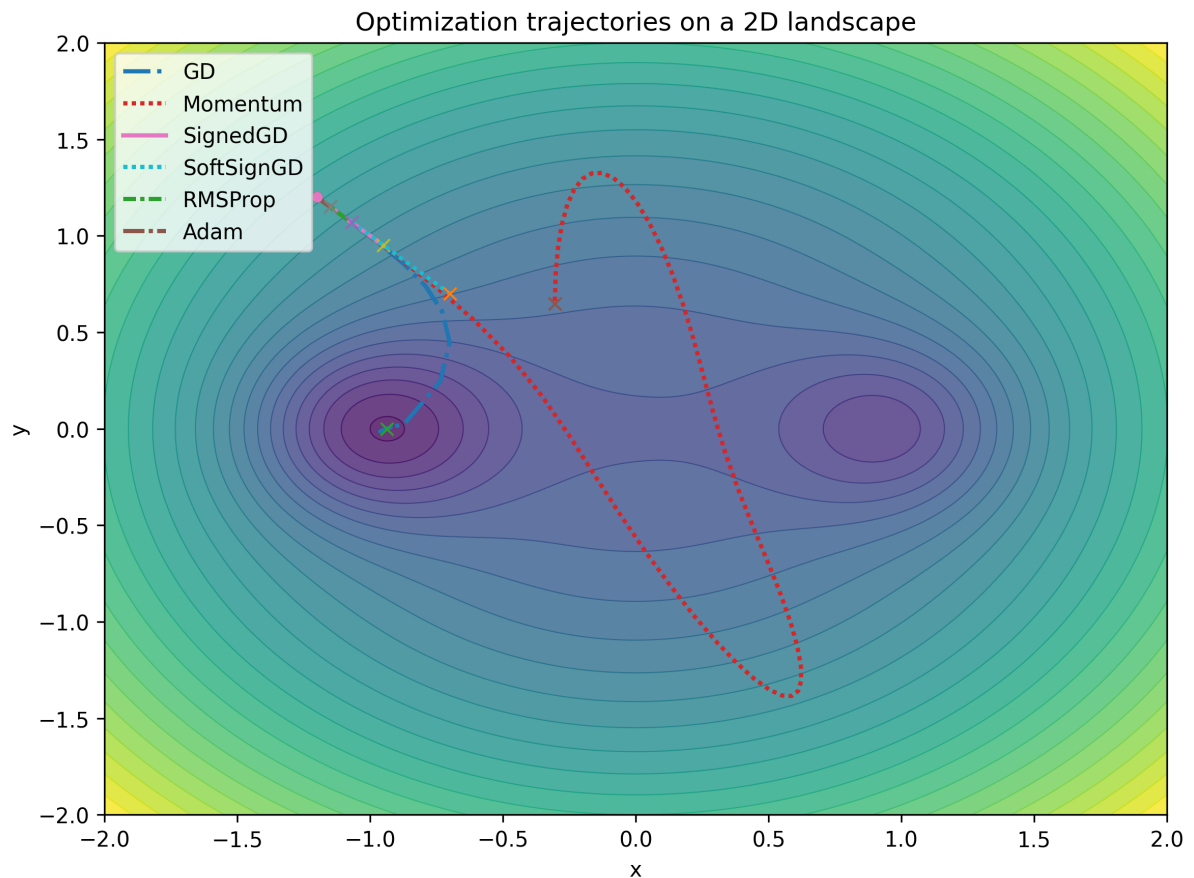


Running for parameters: {'GD': {'lr': 0.05}, 'Momentum': {'lr': 0.01, 'mu': 0.99}, 'SignedGD': {'lr': 0.005}, 'SoftSignGD': {'lr': 0.01, 'tau': 20.0}, 'RMSProp': {'lr': 0.001, 'beta': 0.99, 'eps': 1e-08}, 'Adam': {'lr': 0.001, 'b1': 0.9, 'b2': 0.999, 'eps': 1e-08}}

Final results after 50 steps, start = (-1.2000, 1.2000)

GD: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9363, 0.0000)$
Momentum: $f(x_T) = 0.478565$, at $(x_T, y_T) = (-0.3046, 0.6471)$
SignedGD: $f(x_T) = 1.772496$, at $(x_T, y_T) = (-0.9500, 0.9500)$
SoftSignGD: $f(x_T) = 0.814930$, at $(x_T, y_T) = (-0.7000, 0.7000)$
RMSProp: $f(x_T) = 2.276065$, at $(x_T, y_T) = (-1.0695, 1.0687)$
Adam: $f(x_T) = 2.642452$, at $(x_T, y_T) = (-1.1502, 1.1502)$

Best (lowest loss): GD with $f(x_T) = -2.063090$



```
=====
=====

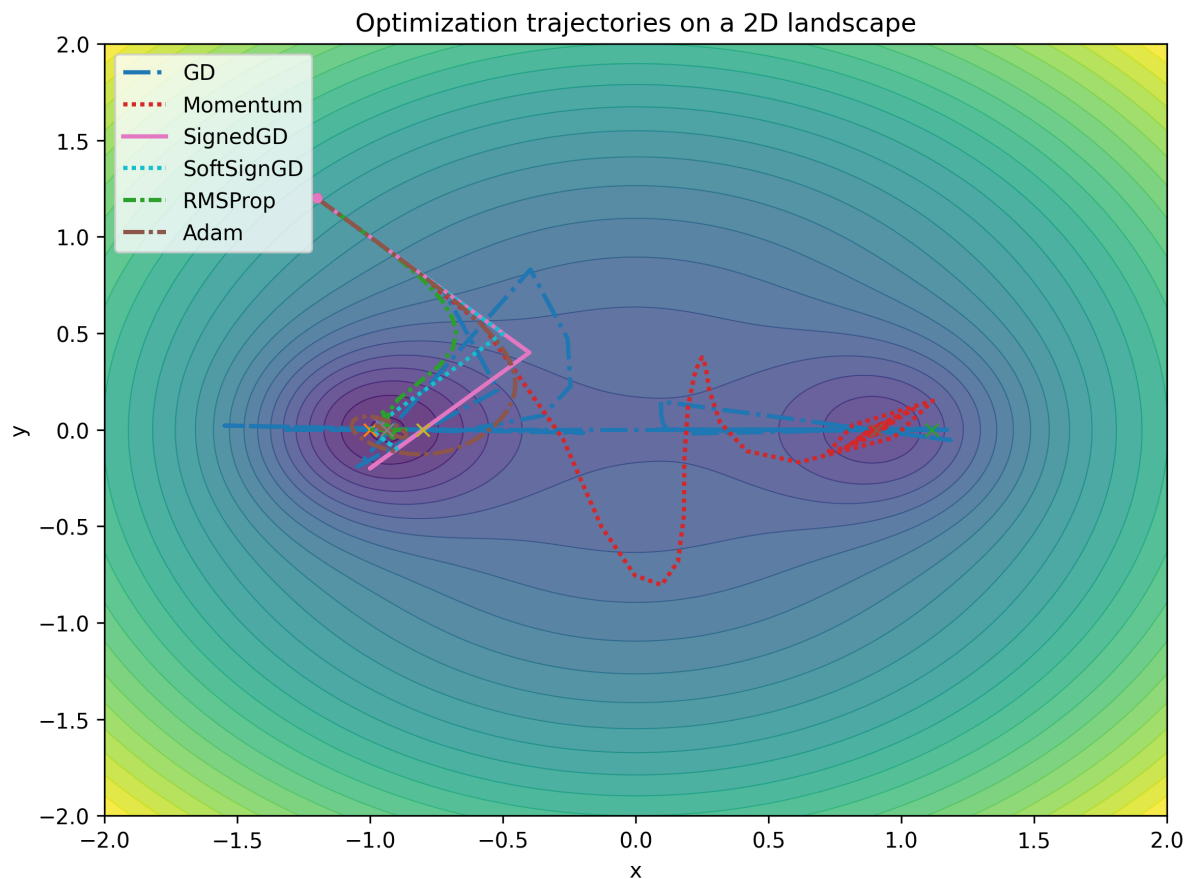
=====
=====
START=(-1.2, 1.2), STEPS=200
=====
=====
```

Running for parameters: {'GD': {'lr': 0.2}, 'Momentum': {'lr': 0.1, 'mu': 0.8}, 'SignedGD': {'lr': 0.2}, 'SoftSignGD': {'lr': 0.1, 'tau': 1.0}, 'RMSProp': {'lr': 0.05, 'beta': 0.9, 'eps': 1e-08}, 'Adam': {'lr': 0.05, 'b1': 0.9, 'b2': 0.99, 'eps': 1e-08}}

Final results after 200 steps, start = (-1.2000, 1.2000)

GD: $f(x_T) = -0.630736$, at $(x_T, y_T) = (1.1146, -0.0000)$
Momentum: $f(x_T) = -1.092732$, at $(x_T, y_T) = (0.9053, 0.0000)$
SignedGD: $f(x_T) = -1.816192$, at $(x_T, y_T) = (-0.8000, 0.0000)$
SoftSignGD: $f(x_T) = -2.000000$, at $(x_T, y_T) = (-1.0000, 0.0000)$
RMSProp: $f(x_T) = -2.044482$, at $(x_T, y_T) = (-0.9605, 0.0247)$
Adam: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9363, -0.0000)$

Best (lowest loss): Adam with $f(x_T) = -2.063090$

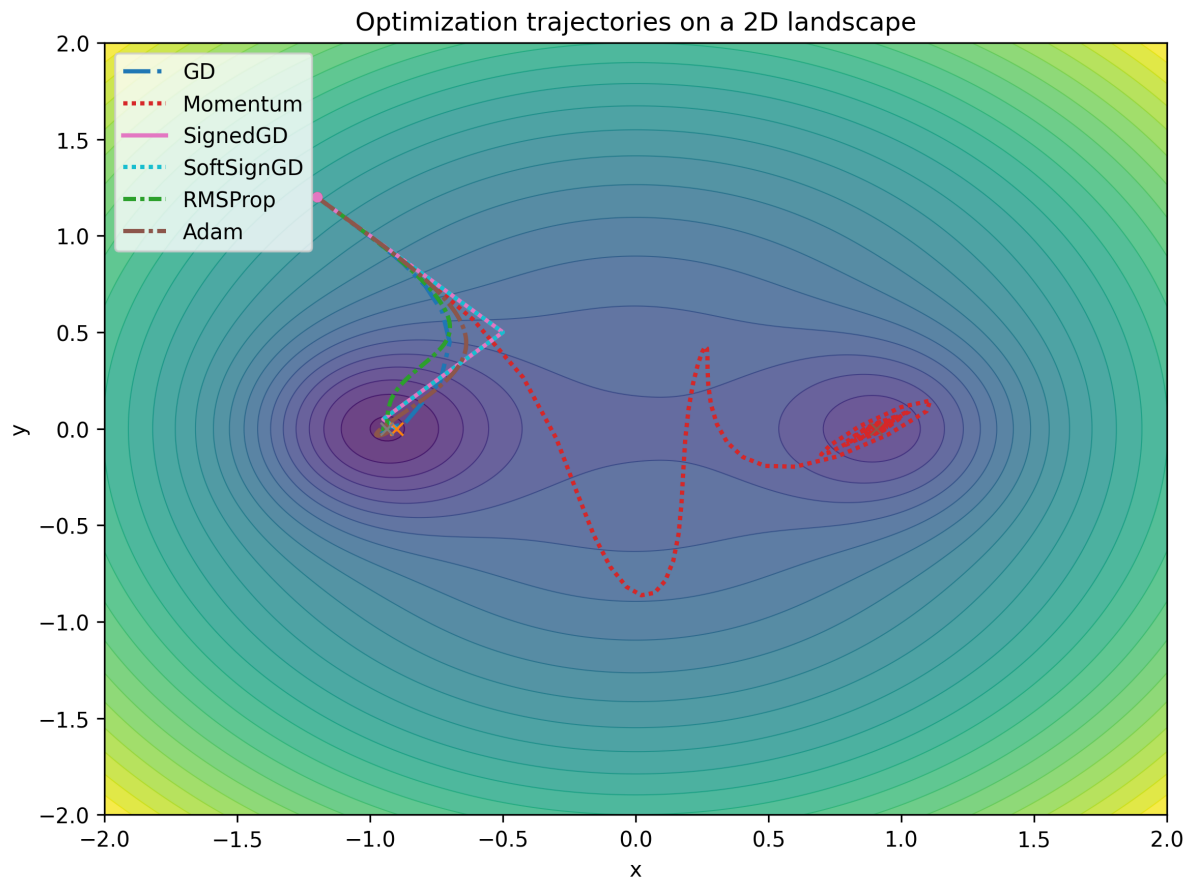


Running for parameters: {'GD': {'lr': 0.05}, 'Momentum': {'lr': 0.03, 'mu': 0.9}, 'SignedGD': {'lr': 0.05}, 'SoftSignGD': {'lr': 0.05, 'tau': 5.0}, 'RMSProp': {'lr': 0.01, 'beta': 0.95, 'eps': 1e-08}, 'Adam': {'lr': 0.01, 'b1': 0.9, 'b2': 0.999, 'eps': 1e-08}}

Final results after 200 steps, start = (-1.2000, 1.2000)

GD: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9363, 0.0000)$
Momentum: $f(x_T) = -1.092732$, at $(x_T, y_T) = (0.9053, -0.0000)$
SignedGD: $f(x_T) = -2.043688$, at $(x_T, y_T) = (-0.9000, -0.0000)$
SoftSignGD: $f(x_T) = -2.043688$, at $(x_T, y_T) = (-0.9000, -0.0000)$
RMSProp: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9363, -0.0000)$
Adam: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9364, 0.0001)$

Best (lowest loss): GD with $f(x_T) = -2.063090$

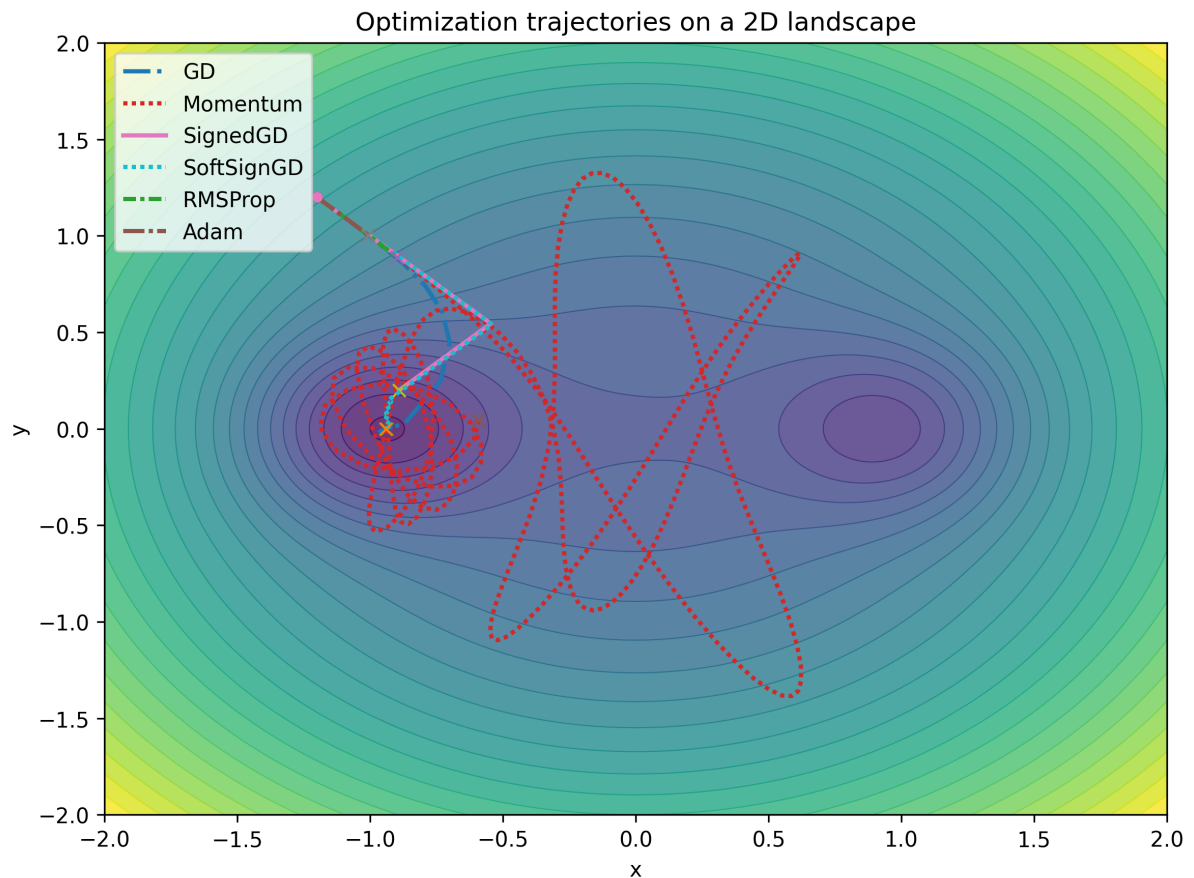


Running for parameters: {'GD': {'lr': 0.05}, 'Momentum': {'lr': 0.01, 'mu': 0.99}, 'SignedGD': {'lr': 0.005}, 'SoftSignGD': {'lr': 0.01, 'tau': 20.0}, 'RMSProp': {'lr': 0.001, 'beta': 0.99, 'eps': 1e-08}, 'Adam': {'lr': 0.001, 'b1': 0.9, 'b2': 0.999, 'eps': 1e-08}}

Final results after 200 steps, start = (-1.2000, 1.2000)

GD: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9363, 0.0000)$
Momentum: $f(x_T) = -0.940259$, at $(x_T, y_T) = (-0.5918, 0.0427)$
SignedGD: $f(x_T) = -1.479898$, at $(x_T, y_T) = (-0.8900, 0.2000)$
SoftSignGD: $f(x_T) = -2.062883$, at $(x_T, y_T) = (-0.9400, -0.0000)$
RMSProp: $f(x_T) = 1.537826$, at $(x_T, y_T) = (-0.8981, 0.8871)$
Adam: $f(x_T) = 2.003145$, at $(x_T, y_T) = (-1.0070, 1.0043)$

Best (lowest loss): GD with $f(x_T) = -2.063090$



```
=====
=====

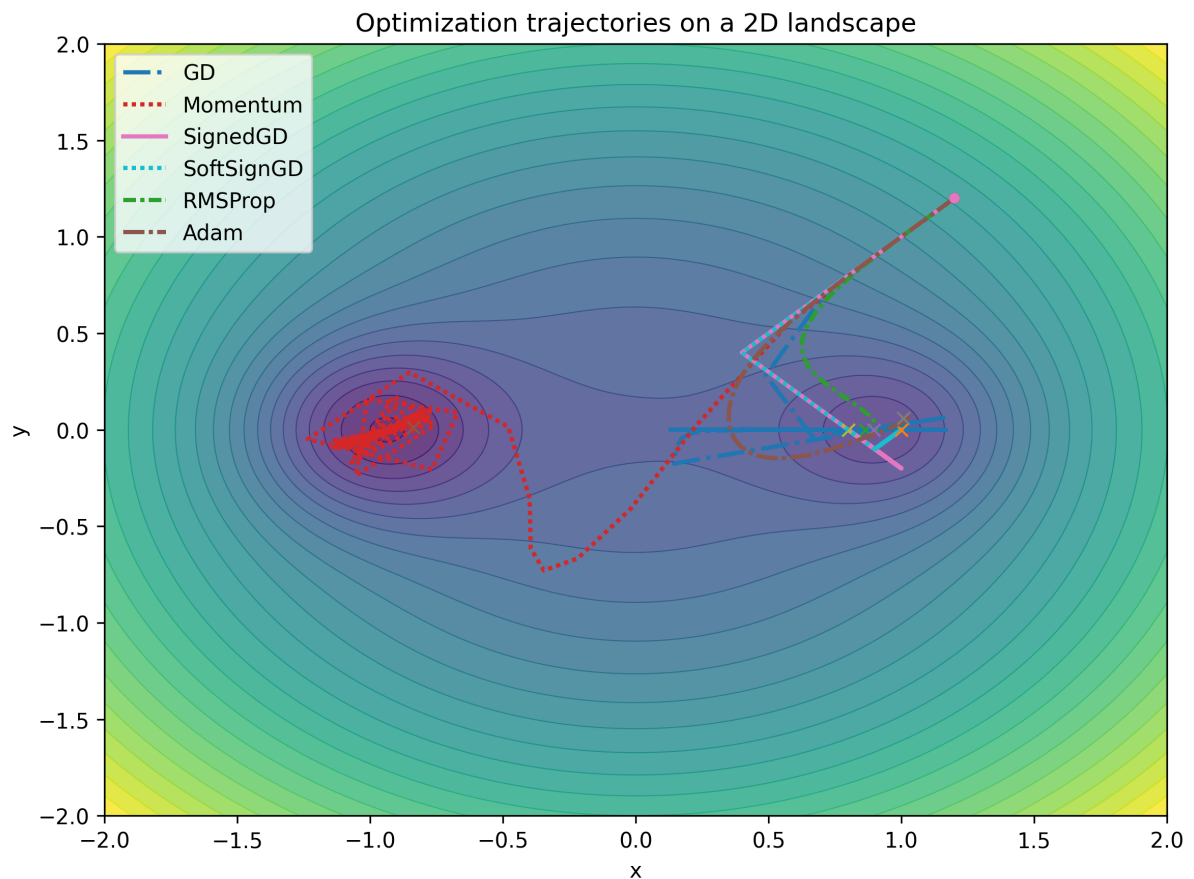
=====
=====
START=(1.2, 1.2), STEPS=50
=====
=====
```

Running for parameters: {'GD': {'lr': 0.2}, 'Momentum': {'lr': 0.1, 'mu': 0.8}, 'SignedGD': {'lr': 0.2}, 'SoftSignGD': {'lr': 0.1, 'tau': 1.0}, 'RMSProp': {'lr': 0.05, 'beta': 0.9, 'eps': 1e-08}, 'Adam': {'lr': 0.05, 'b1': 0.9, 'b2': 0.99, 'eps': 1e-08}}

Final results after 50 steps, start = (1.2000, 1.2000)

GD:	$f(x_T) = -1.076854$,	at $(x_T, y_T) = (0.8640, 0.0000)$
Momentum:	$f(x_T) = -1.926684$,	at $(x_T, y_T) = (-0.8379, 0.0114)$
SignedGD:	$f(x_T) = -0.997462$,	at $(x_T, y_T) = (0.8000, 0.0000)$
SoftSignGD:	$f(x_T) = -1.000000$,	at $(x_T, y_T) = (1.0000, 0.0000)$
RMSProp:	$f(x_T) = -1.091951$,	at $(x_T, y_T) = (0.8963, 0.0000)$
Adam:	$f(x_T) = -0.946084$,	at $(x_T, y_T) = (1.0075, 0.0593)$

Best (lowest loss): Momentum with $f(x_T) = -1.926684$

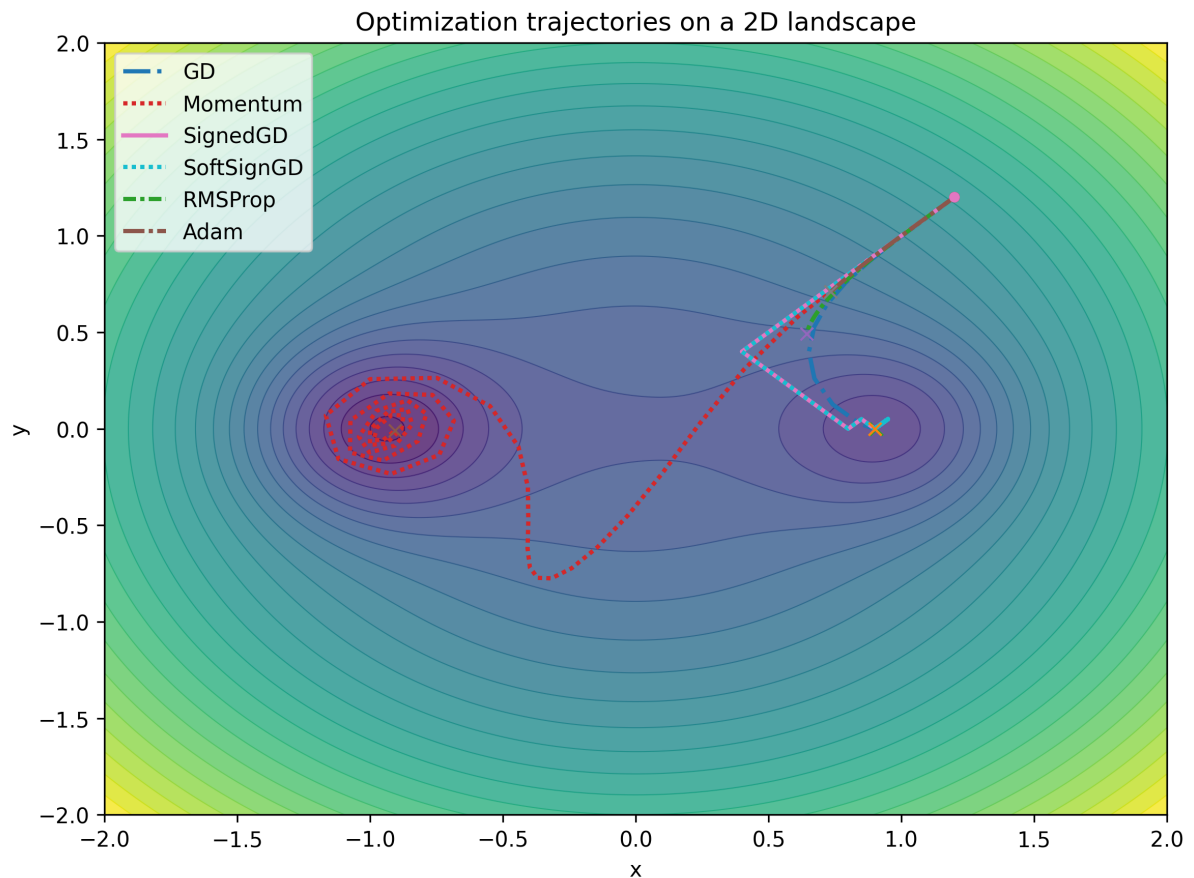


Running for parameters: {'GD': {'lr': 0.05}, 'Momentum': {'lr': 0.03, 'mu': 0.9}, 'SignedGD': {'lr': 0.05}, 'SoftSignGD': {'lr': 0.05, 'tau': 5.0}, 'RMSProp': {'lr': 0.01, 'beta': 0.95, 'eps': 1e-08}, 'Adam': {'lr': 0.01, 'b1': 0.9, 'b2': 0.999, 'eps': 1e-08}}

Final results after 50 steps, start = (1.2000, 1.2000)

GD: $f(x_T) = -1.092732$, at $(x_T, y_T) = (0.9053, -0.0000)$
Momentum: $f(x_T) = -2.048663$, at $(x_T, y_T) = (-0.9064, -0.0089)$
SignedGD: $f(x_T) = -1.092459$, at $(x_T, y_T) = (0.9000, -0.0000)$
SoftSignGD: $f(x_T) = -1.092459$, at $(x_T, y_T) = (0.9000, -0.0000)$
RMSProp: $f(x_T) = 0.342836$, at $(x_T, y_T) = (0.6440, 0.4929)$
Adam: $f(x_T) = 0.929916$, at $(x_T, y_T) = (0.7346, 0.7097)$

Best (lowest loss): Momentum with $f(x_T) = -2.048663$

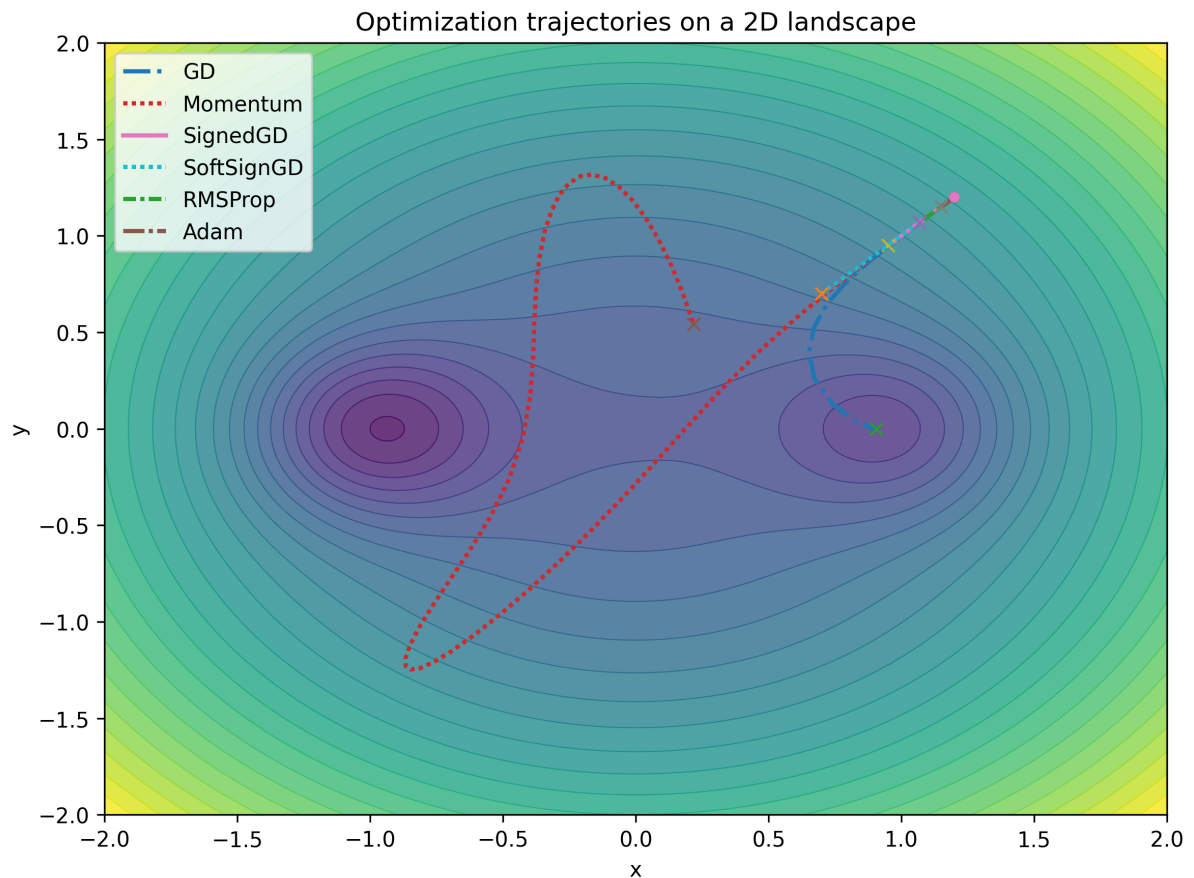


Running for parameters: {'GD': {'lr': 0.05}, 'Momentum': {'lr': 0.01, 'mu': 0.99}, 'SignedGD': {'lr': 0.005}, 'SoftSignGD': {'lr': 0.01, 'tau': 20.0}, 'RMSProp': {'lr': 0.001, 'beta': 0.99, 'eps': 1e-08}, 'Adam': {'lr': 0.001, 'b1': 0.9, 'b2': 0.999, 'eps': 1e-08}}

Final results after 50 steps, start = (1.2000, 1.2000)

GD: $f(x_T) = -1.092732$, at $(x_T, y_T) = (0.9053, -0.0000)$
Momentum: $f(x_T) = 0.319981$, at $(x_T, y_T) = (0.2176, 0.5427)$
SignedGD: $f(x_T) = 1.783331$, at $(x_T, y_T) = (0.9500, 0.9500)$
SoftSignGD: $f(x_T) = 0.869953$, at $(x_T, y_T) = (0.7000, 0.7000)$
RMSProp: $f(x_T) = 2.279944$, at $(x_T, y_T) = (1.0695, 1.0689)$
Adam: $f(x_T) = 2.643691$, at $(x_T, y_T) = (1.1503, 1.1502)$

Best (lowest loss): GD with $f(x_T) = -1.092732$



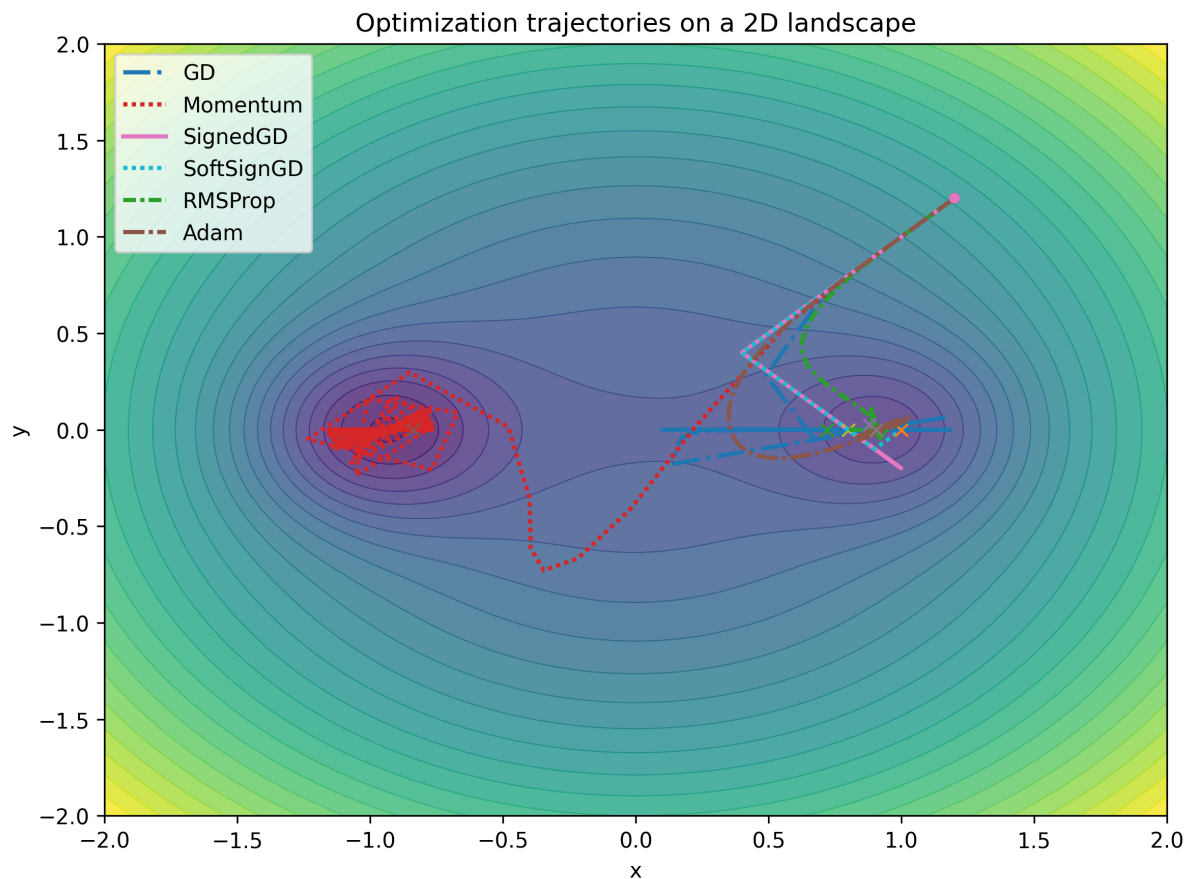
```
=====
START=(1.2, 1.2), STEPS=200
=====
```

```
Running for parameters: {'GD': {'lr': 0.2}, 'Momentum': {'lr': 0.1, 'm
u': 0.8}, 'SignedGD': {'lr': 0.2}, 'SoftSignGD': {'lr': 0.1, 'tau': 1.
0}, 'RMSProp': {'lr': 0.05, 'beta': 0.9, 'eps': 1e-08}, 'Adam': {'lr':
0.05, 'b1': 0.9, 'b2': 0.99, 'eps': 1e-08}}
```

```
Final results after 200 steps, start = (1.2000, 1.2000)
```

```
GD: f(x_T) = -0.834286, at (x_T, y_T) = ( 0.7205, 0.0000)
Momentum: f(x_T) = -1.925176, at (x_T, y_T) = (-0.8366, 0.0001)
SignedGD: f(x_T) = -0.997462, at (x_T, y_T) = ( 0.8000, 0.0000)
SoftSignGD: f(x_T) = -1.000000, at (x_T, y_T) = ( 1.0000, 0.0000)
RMSProp: f(x_T) = -1.079542, at (x_T, y_T) = ( 0.8792, 0.0256)
Adam: f(x_T) = -1.092732, at (x_T, y_T) = ( 0.9054, 0.0000)
```

```
Best (lowest loss): Momentum with f(x_T) = -1.925176
```

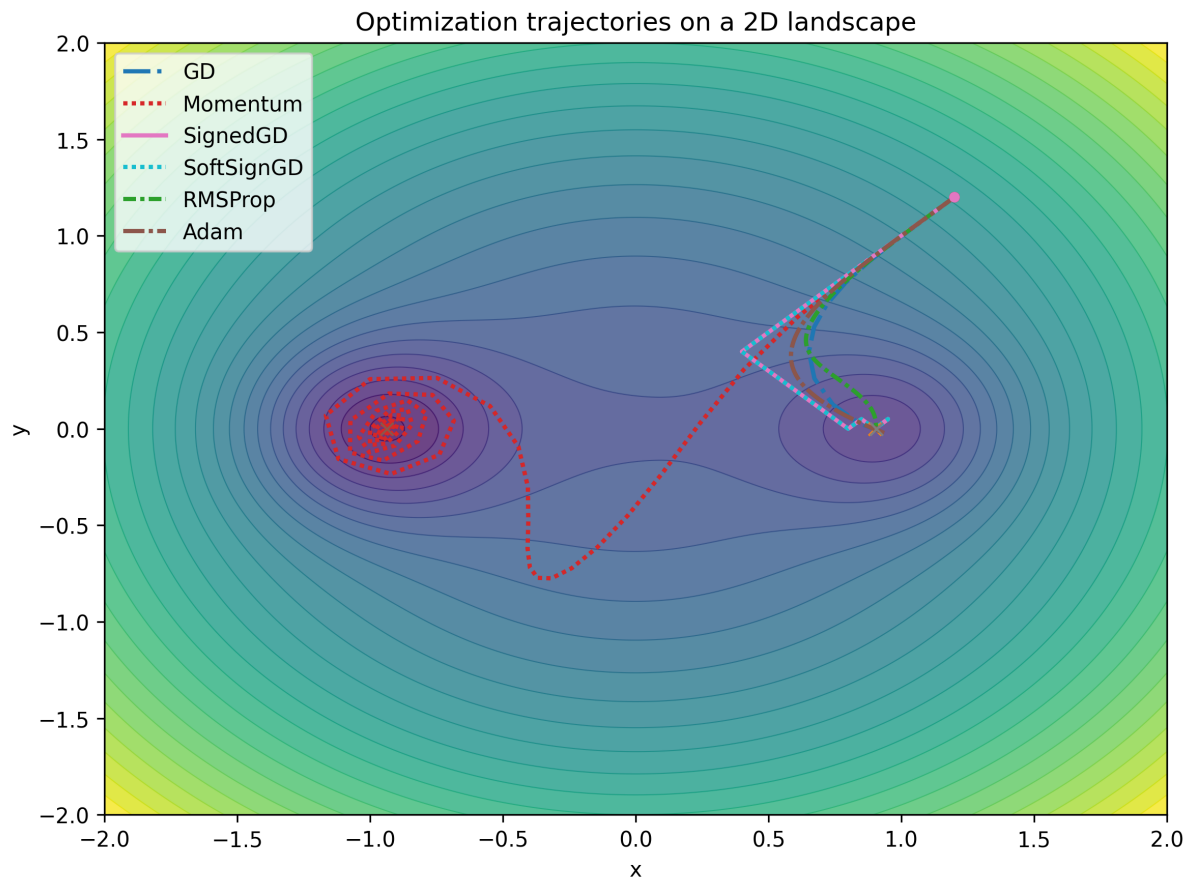


Running for parameters: {'GD': {'lr': 0.05}, 'Momentum': {'lr': 0.03, 'mu': 0.9}, 'SignedGD': {'lr': 0.05}, 'SoftSignGD': {'lr': 0.05, 'tau': 5.0}, 'RMSProp': {'lr': 0.01, 'beta': 0.95, 'eps': 1e-08}, 'Adam': {'lr': 0.01, 'b1': 0.9, 'b2': 0.999, 'eps': 1e-08}}

Final results after 200 steps, start = (1.2000, 1.2000)

GD: $f(x_T) = -1.092732$, at $(x_T, y_T) = (0.9053, -0.0000)$
Momentum: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9363, -0.0000)$
SignedGD: $f(x_T) = -1.092459$, at $(x_T, y_T) = (0.9000, -0.0000)$
SoftSignGD: $f(x_T) = -1.092459$, at $(x_T, y_T) = (0.9000, -0.0000)$
RMSProp: $f(x_T) = -1.092732$, at $(x_T, y_T) = (0.9053, 0.0000)$
Adam: $f(x_T) = -1.092714$, at $(x_T, y_T) = (0.9064, -0.0009)$

Best (lowest loss): Momentum with $f(x_T) = -2.063090$

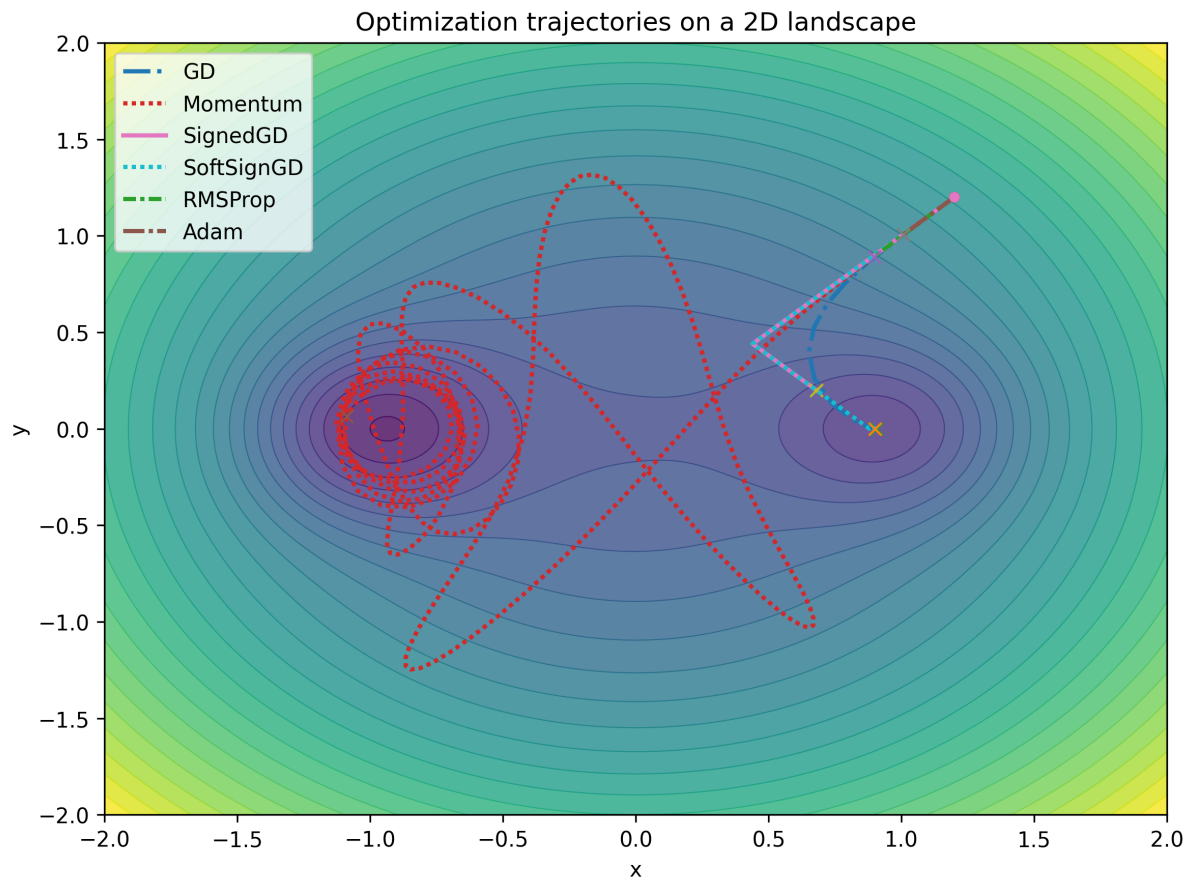


Running for parameters: {'GD': {'lr': 0.05}, 'Momentum': {'lr': 0.01, 'mu': 0.99}, 'SignedGD': {'lr': 0.005}, 'SoftSignGD': {'lr': 0.01, 'tau': 20.0}, 'RMSProp': {'lr': 0.001, 'beta': 0.99, 'eps': 1e-08}, 'Adam': {'lr': 0.001, 'b1': 0.9, 'b2': 0.999, 'eps': 1e-08}}

Final results after 200 steps, start = (1.2000, 1.2000)

GD: $f(x_T) = -1.092732$, at $(x_T, y_T) = (0.9053, -0.0000)$
Momentum: $f(x_T) = -1.633242$, at $(x_T, y_T) = (-1.0888, 0.0654)$
SignedGD: $f(x_T) = -0.478926$, at $(x_T, y_T) = (0.6800, 0.2000)$
SoftSignGD: $f(x_T) = -1.092459$, at $(x_T, y_T) = (0.9000, -0.0000)$
RMSProp: $f(x_T) = 1.562643$, at $(x_T, y_T) = (0.8977, 0.8904)$
Adam: $f(x_T) = 2.011576$, at $(x_T, y_T) = (1.0070, 1.0052)$

Best (lowest loss): Momentum with $f(x_T) = -1.633242$



```
=====
=====

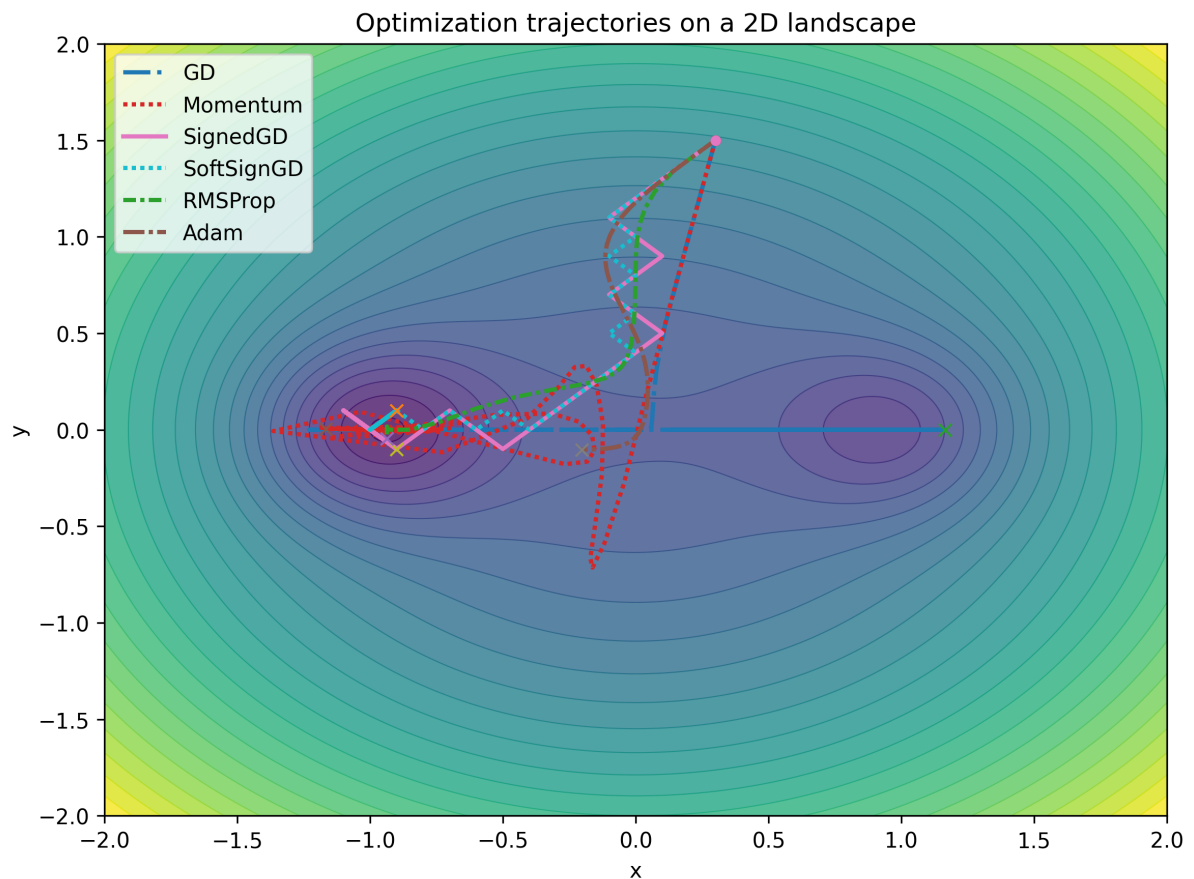
=====
=====
START=(0.3, 1.5), STEPS=50
=====
=====
```

Running for parameters: {'GD': {'lr': 0.2}, 'Momentum': {'lr': 0.1, 'mu': 0.8}, 'SignedGD': {'lr': 0.2}, 'SoftSignGD': {'lr': 0.1, 'tau': 1.0}, 'RMSProp': {'lr': 0.05, 'beta': 0.9, 'eps': 1e-08}, 'Adam': {'lr': 0.05, 'b1': 0.9, 'b2': 0.99, 'eps': 1e-08}}

Final results after 50 steps, start = (0.3000, 1.5000)

GD: $f(x_T) = -0.393347$, at $(x_T, y_T) = (1.1640, 0.0000)$
Momentum: $f(x_T) = -1.245949$, at $(x_T, y_T) = (-1.1672, 0.0042)$
SignedGD: $f(x_T) = -1.894512$, at $(x_T, y_T) = (-0.9000, -0.1000)$
SoftSignGD: $f(x_T) = -1.894512$, at $(x_T, y_T) = (-0.9000, 0.1000)$
RMSProp: $f(x_T) = -2.029809$, at $(x_T, y_T) = (-0.9361, -0.0462)$
Adam: $f(x_T) = -0.068637$, at $(x_T, y_T) = (-0.2035, -0.1028)$

Best (lowest loss): RMSProp with $f(x_T) = -2.029809$

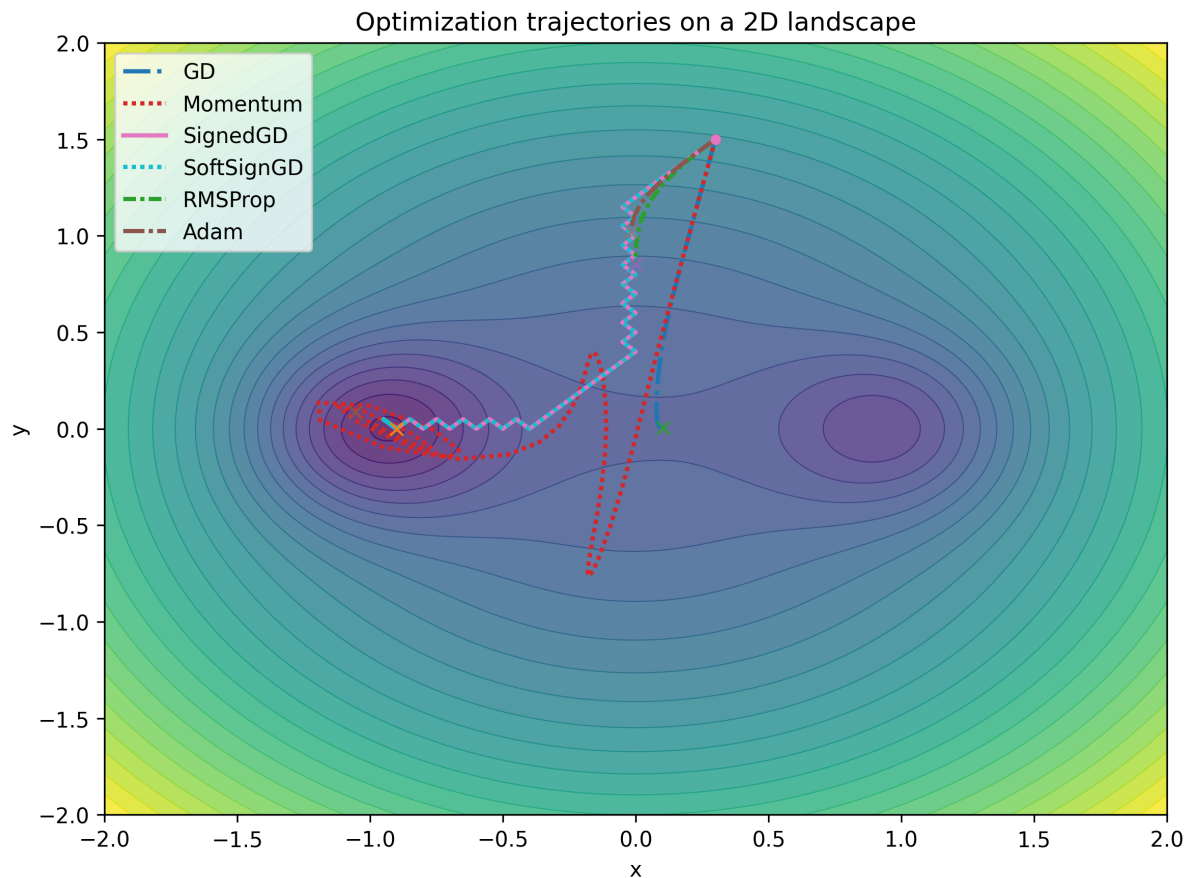


Running for parameters: {'GD': {'lr': 0.05}, 'Momentum': {'lr': 0.03, 'mu': 0.9}, 'SignedGD': {'lr': 0.05}, 'SoftSignGD': {'lr': 0.05, 'tau': 5.0}, 'RMSProp': {'lr': 0.01, 'beta': 0.95, 'eps': 1e-08}, 'Adam': {'lr': 0.01, 'b1': 0.9, 'b2': 0.999, 'eps': 1e-08}}

Final results after 50 steps, start = (0.3000, 1.5000)

GD: $f(x_T) = -0.032065$, at $(x_T, y_T) = (0.1042, 0.0035)$
Momentum: $f(x_T) = -1.725944$, at $(x_T, y_T) = (-1.0546, 0.0871)$
SignedGD: $f(x_T) = -2.043688$, at $(x_T, y_T) = (-0.9000, -0.0000)$
SoftSignGD: $f(x_T) = -2.043688$, at $(x_T, y_T) = (-0.9000, -0.0000)$
RMSProp: $f(x_T) = 0.729051$, at $(x_T, y_T) = (-0.0002, 0.8544)$
Adam: $f(x_T) = 1.046369$, at $(x_T, y_T) = (-0.0169, 1.0229)$

Best (lowest loss): SignedGD with $f(x_T) = -2.043688$

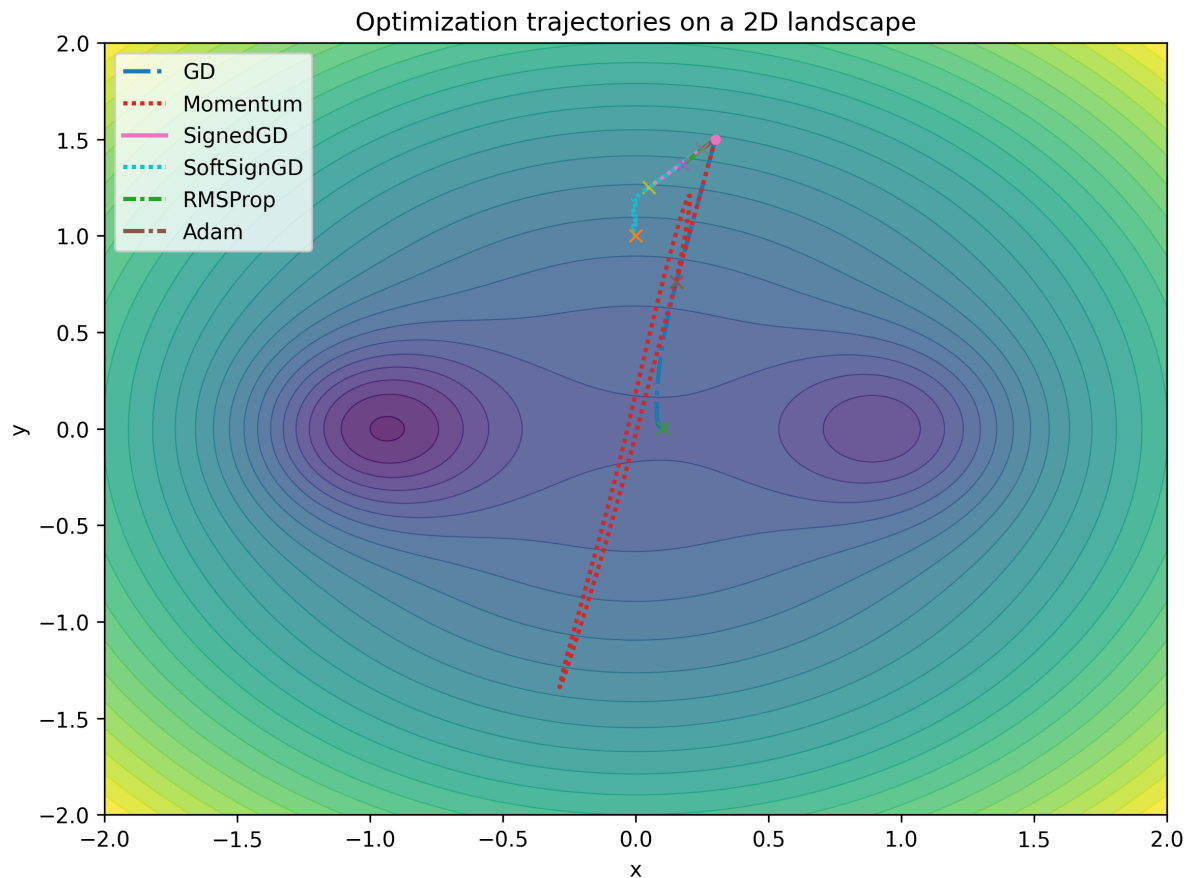


Running for parameters: {'GD': {'lr': 0.05}, 'Momentum': {'lr': 0.01, 'mu': 0.99}, 'SignedGD': {'lr': 0.005}, 'SoftSignGD': {'lr': 0.01, 'tau': 20.0}, 'RMSProp': {'lr': 0.001, 'beta': 0.99, 'eps': 1e-08}, 'Adam': {'lr': 0.001, 'b1': 0.9, 'b2': 0.999, 'eps': 1e-08}}

Final results after 50 steps, start = (0.3000, 1.5000)

GD: $f(x_T) = -0.032065$, at $(x_T, y_T) = (0.1042, 0.0035)$
Momentum: $f(x_T) = 0.599923$, at $(x_T, y_T) = (0.1526, 0.7615)$
SignedGD: $f(x_T) = 1.564986$, at $(x_T, y_T) = (0.0500, 1.2500)$
SoftSignGD: $f(x_T) = 0.999773$, at $(x_T, y_T) = (-0.0000, 1.0000)$
RMSProp: $f(x_T) = 1.906000$, at $(x_T, y_T) = (0.1790, 1.3689)$
Adam: $f(x_T) = 2.166119$, at $(x_T, y_T) = (0.2511, 1.4502)$

Best (lowest loss): GD with $f(x_T) = -0.032065$



```
=====
=====

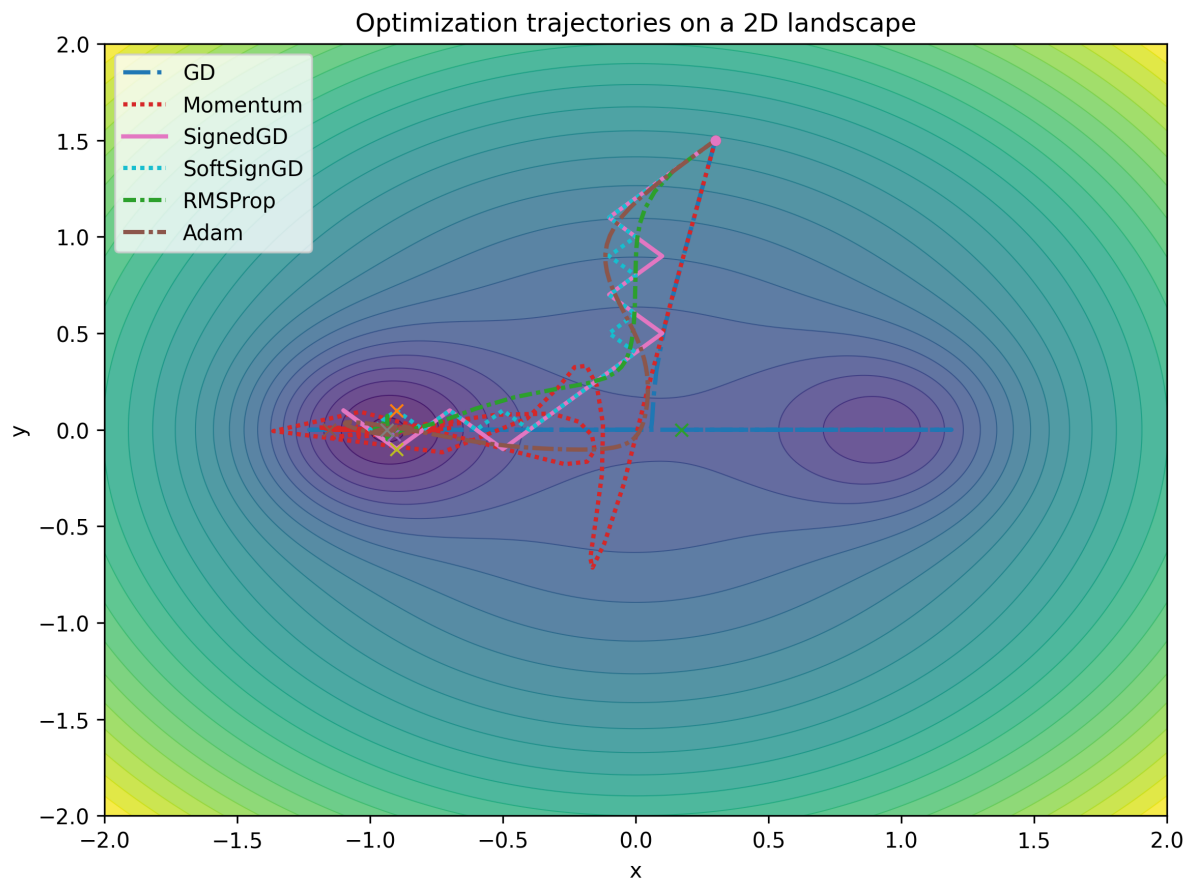
=====
=====
START=(0.3, 1.5), STEPS=200
=====
=====
```

Running for parameters: {'GD': {'lr': 0.2}, 'Momentum': {'lr': 0.1, 'mu': 0.8}, 'SignedGD': {'lr': 0.2}, 'SoftSignGD': {'lr': 0.1, 'tau': 1.0}, 'RMSProp': {'lr': 0.05, 'beta': 0.9, 'eps': 1e-08}, 'Adam': {'lr': 0.05, 'b1': 0.9, 'b2': 0.99, 'eps': 1e-08}}

Final results after 200 steps, start = (0.3000, 1.5000)

GD: $f(x_T) = -0.038519$, at $(x_T, y_T) = (0.1725, 0.0000)$
Momentum: $f(x_T) = -1.332109$, at $(x_T, y_T) = (-1.1541, 0.0000)$
SignedGD: $f(x_T) = -1.894512$, at $(x_T, y_T) = (-0.9000, -0.1000)$
SoftSignGD: $f(x_T) = -1.894512$, at $(x_T, y_T) = (-0.9000, 0.1000)$
RMSProp: $f(x_T) = -2.043318$, at $(x_T, y_T) = (-0.9105, -0.0254)$
Adam: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9364, -0.0000)$

Best (lowest loss): Adam with $f(x_T) = -2.063090$

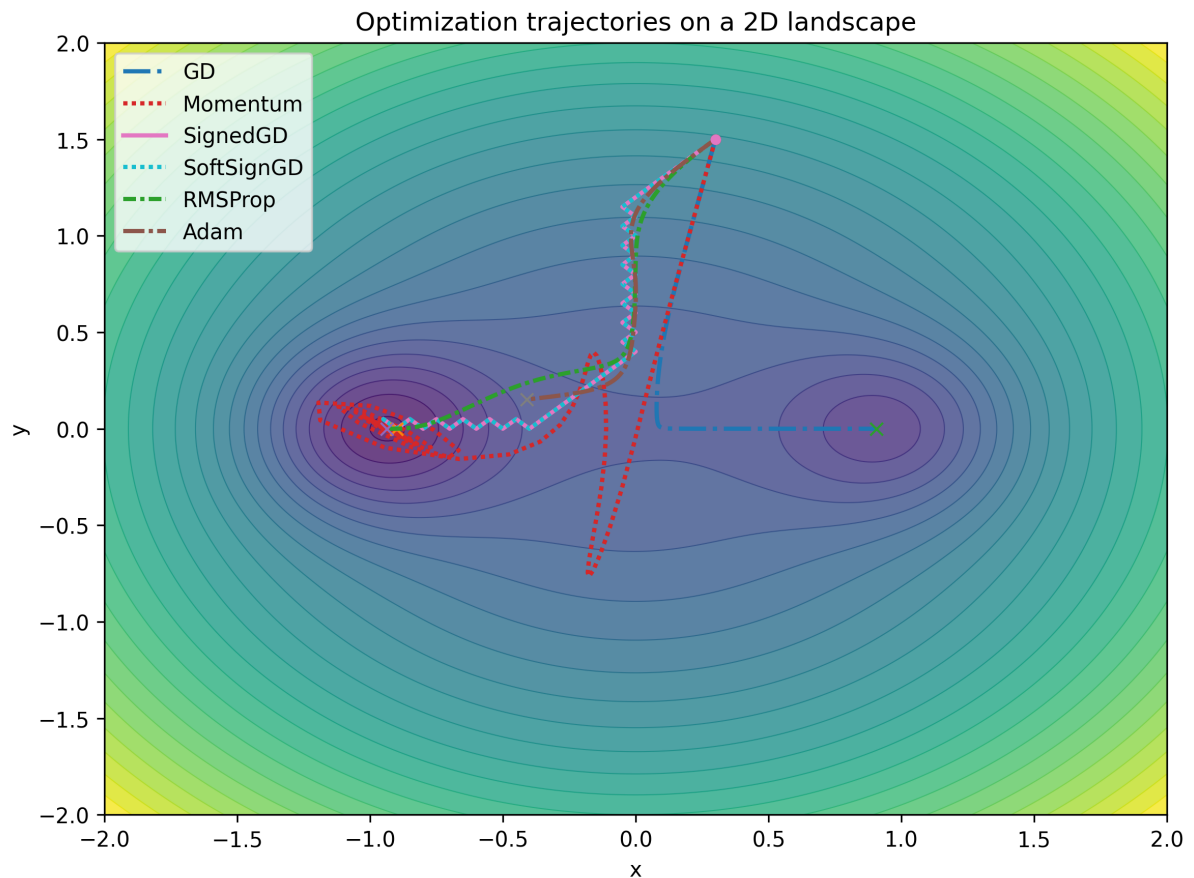


Running for parameters: {'GD': {'lr': 0.05}, 'Momentum': {'lr': 0.03, 'mu': 0.9}, 'SignedGD': {'lr': 0.05}, 'SoftSignGD': {'lr': 0.05, 'tau': 5.0}, 'RMSProp': {'lr': 0.01, 'beta': 0.95, 'eps': 1e-08}, 'Adam': {'lr': 0.01, 'b1': 0.9, 'b2': 0.999, 'eps': 1e-08}}

Final results after 200 steps, start = (0.3000, 1.5000)

GD: $f(x_T) = -1.092732$, at $(x_T, y_T) = (0.9053, 0.0000)$
 Momentum: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9363, 0.0000)$
 SignedGD: $f(x_T) = -2.043688$, at $(x_T, y_T) = (-0.9000, -0.0000)$
 SoftSignGD: $f(x_T) = -2.043688$, at $(x_T, y_T) = (-0.9000, -0.0000)$
 RMSProp: $f(x_T) = -2.063090$, at $(x_T, y_T) = (-0.9363, 0.0000)$
 Adam: $f(x_T) = -0.279670$, at $(x_T, y_T) = (-0.4107, 0.1514)$

Best (lowest loss): RMSProp with $f(x_T) = -2.063090$

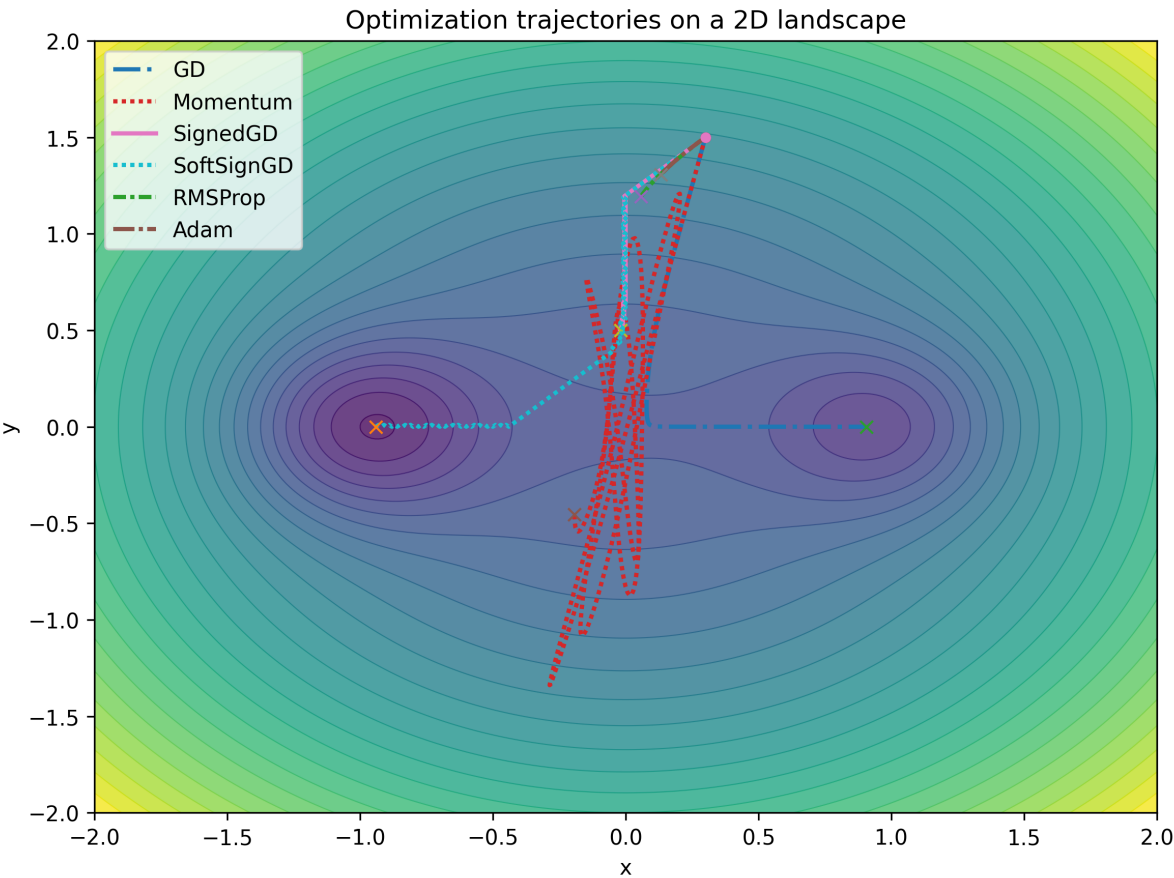


Running for parameters: {'GD': {'lr': 0.05}, 'Momentum': {'lr': 0.01, 'mu': 0.99}, 'SignedGD': {'lr': 0.005}, 'SoftSignGD': {'lr': 0.01, 'tau': 20.0}, 'RMSProp': {'lr': 0.001, 'beta': 0.99, 'eps': 1e-08}, 'Adam': {'lr': 0.001, 'b1': 0.9, 'b2': 0.999, 'eps': 1e-08}}

Final results after 200 steps, start = (0.3000, 1.5000)

GD:	$f(x_T) = -1.092732$	at $(x_T, y_T) = (0.9053, 0.0000)$
Momentum:	$f(x_T) = 0.202876$	at $(x_T, y_T) = (-0.1932, -0.4552)$
SignedGD:	$f(x_T) = 0.240186$	at $(x_T, y_T) = (-0.0200, 0.5000)$
SoftSignGD:	$f(x_T) = -2.062883$	at $(x_T, y_T) = (-0.9400, -0.0000)$
RMSProp:	$f(x_T) = 1.429096$	at $(x_T, y_T) = (0.0576, 1.1941)$
Adam:	$f(x_T) = 1.721732$	at $(x_T, y_T) = (0.1313, 1.3056)$

Best (lowest loss): SoftSignGD with $f(x_T) = -2.062883$



=====

=====

3. For each optimizer, I am going to go through 3 rounds of different hyper-parameters to see the best performance for each. The findings of each setting is found in the output from the cell above. From the graphs we can see whether the optimizer converged left or right along with the final objective. Based on the results, here are the best settings for each optimizer:

Optimizer	Parameters	Final Result
GD	lr = 0.05	f = -1.7396

Momentum	lr = 0.03, mu = 0.9	f = -1.7338
SignedGD	lr = 0.03	f = -1.7335
SoftSignedGD	lr = 0.10, tau = 5.0	f = -1.6.315
RMSProp	lr = 0.03, beta = 0.9, eps = 1e-8	f = -1.4161
Adam	lr = 0.05, beta = 0.9, eps = 1e-8	f = -1.4161

I chose these settings because after multiple runs with many different parameters, these parameters seem to perform the best with the best convergences for each optimizer. Overall, momentum did the best out of all the optimizers. It was able to settle deeper and was able to reliably settle despite the different starts. Momentum accumulates past gradients, this helps momentum continue to move consistently to the correct well and stabilize there.

Problem 3

1. I used 4 different architectures to test an arbitrary number of layers for the mlp model. I use the baseline number of layers that is provided in the starter code, a higher number of hidden units, more number of layers, and more layers with a high number of hidden units. The baseline layer shows a 1% of loss after training with training data and test data. As we progress with each kind of architecture, the loss seems to converge to 0% much quicker. With the last layer that has more than 1 layer with a high number of hidden units converging to 0% loss by step 600.
2. I performed the training on sigmas 10^{-5} , 10^{-1} , and 10. The results on sigma 10^{-5} data make the model seem to immediately have learned the data by step 100. For sigma 10^{-1} , the results show that the model had the healthiest curve of the 3 with loss being converged by step 300. For sigma 10, the model seems to "explode" with loss being a NaN. These results show that if sigma is too small then the model seems to collapse to a single point or line and may not learn. If Sigma is too big, then there exist numerical instability. To solve this issue, initialization can be done by scaling sigma.
3. Sigmoid is relatively harder to train than ReLU due to the computationally expensive evaluation of the sigmoid function. Due to the nature of the sigmoid, the vanishing gradient problem becomes a primary leading cause in the difficulty of training a model using the sigmoid. After several attempts with several different initialization parameters such as bias = 0, adjusting learning rates from 1e-2 to 2e-2 to 5e-2, adjusting the scale from 1e-1 to 1e-

2 to $2e-2$, I could not improve the model's loss any better. The final results I got was test loss = 0.66 and training loss is 0.43.

4.

- Huber loss is less sensitive to outliers due to the use of quadratic and linear penalty for different errors. When an outlier is large, it will use the linear penalty which stops a single massive error from dominating the process and affecting the overall model accuracy.
- When adding in the Huber loss and adding outliers to the data, Huber loss performs better than the MSE loss. MSE loss gets to train loss of 7.15 and 0.89 test loss with the starter code including the outliers of 10 for the first 5 elements. As for the Huber loss, it got a train loss of 0.82 and test loss of 0.21. This shows that Huber Loss performed a lot better than MSE for the outliers.